

Excel & Pivot Tables — Complete Reference

EY Probation Training — Week 1. This is the topic to over-prepare: everything else on the roadmap is closer to your existing work.

1. Absolute vs Relative References

The single most common source of wrong answers in Excel assessments.

Form	Meaning	On copy
A1	Relative	Both row and column shift
\$A\$1	Absolute	Nothing shifts
A\$1	Row locked	Column shifts, row fixed
\$A1	Column locked	Row shifts, column fixed

F4 cycles through all four while editing. Press it repeatedly on a selected reference.

When you need each: a lookup table range must be `$A$2:$D$100` so it doesn't drift as you fill down. A multiplication table needs `A$1` across and `$A1` down. Getting this wrong produces answers that are right in the first cell and wrong everywhere below it — which is exactly what graders look for.

2. Core Functions

Logical

```
=IF(condition, value_if_true, value_if_false)
=IF(AND(A2>100, B2="West"), "Yes", "No")
=IF(OR(A2="X", A2="Y"), 1, 0)
=IFS(A2>90,"A", A2>75,"B", A2>60,"C", TRUE,"D") ' avoids nested IF
=IFERROR(formula, "fallback") ' wrap the WHOLE formula
=IFNA(formula, "not found") ' only catches #N/A
```

Aggregation with conditions

```
=SUMIF(range, criteria, [sum_range])
=SUMIFS(sum_range, crit_range1, crit1, crit_range2, crit2, ...)
=COUNTIF(range, criteria)
=COUNTIFS(crit_range1, crit1, crit_range2, crit2, ...)
=AVERAGEIFS(avg_range, crit_range1, crit1, ...)
=MAXIFS(max_range, crit_range1, crit1, ...)
=MINIFS(min_range, crit_range1, crit1, ...)
```

Argument order trap: `SUMIF` puts the sum range LAST; `SUMIFS` puts it FIRST. This inconsistency is deliberately tested.

Criteria syntax — comparison operators go inside quotes:

```
=SUMIFS(C:C, B:B, ">100") ' greater than 100
=SUMIFS(C:C, B:B, ">="&E1) ' compare to a cell: concatenate
=COUNTIF(A:A, "*west*") ' wildcard: contains "west"
=COUNTIF(A:A, "?at") ' single-char wildcard: cat, bat, hat
=COUNTIFS(A:A, "<>"&"") ' non-blank
```

Text

```
=TRIM(text) ' removes leading/trailing AND doubled inner spaces
=CLEAN(text) ' strips non-printable characters
=UPPER / LOWER / PROPER(text)
=LEFT(text, n) =RIGHT(text, n) =MID(text, start, n)
=LEN(text)
=FIND("x", text) ' case-SENSITIVE, errors if not found
=SEARCH("x", text) ' case-INSensitive, allows wildcards
=SUBSTITUTE(text, old, new, [instance])
=REPLACE(text, start, n_chars, new)
=CONCAT(A1, " ", B1) or =A1&" "&B1
=TEXTJOIN(" ", TRUE, A1:A10) ' TRUE = skip blanks
=TEXT(value, "0.00%") ' format a number as text
```

`TRIM` + `UPPER` is the standard cleanup combination for dirty categorical columns.

Dates

```

=TODAY() =NOW()
=YEAR(d) =MONTH(d) =DAY(d)
=EOMONTH(d, 0) ' last day of d's month; 0 = current, -1 = previous
=EDATE(d, 3) ' same day, 3 months later
=DATEDIF(start, end, "Y") ' "Y" years, "M" months, "D" days
=NETWORKDAYS(start, end, [holidays])
=WEEKDAY(d, 2) ' 2 = Monday is 1
=DATE(year, month, day)

```

Quarter label: `=YEAR(A2)&"-Q"&ROUNDUP(MONTH(A2)/3,0)`

Dates are serial numbers (1 = 1 Jan 1900). Subtracting two dates gives days. If a date shows as a number, it's a formatting issue, not a data issue.

Math and rounding

```

=ROUND(x, 2) =ROUNDUP(x, 0) =ROUNDDOWN(x, 0)
=MROUND(x, 5) ' nearest multiple of 5
=INT(x) =TRUNC(x, 2)
=ABS(x) =MOD(a, b) =POWER(a, b) =SQRT(x)
=SUMPRODUCT(range1, range2, ...)
=SUBTOTAL(9, range) ' 9 = SUM, ignores filtered-out rows
=AGGREGATE(9, 5, range) ' like SUBTOTAL but can also ignore errors

```

SUBTOTAL vs SUM: `SUM` includes hidden rows; `SUBTOTAL(9,...)` respects the filter. On a filtered dashboard you almost always want `SUBTOTAL`. Function codes: 1=AVERAGE, 2=COUNT, 3=COUNTA, 4=MAX, 5=MIN, 9=SUM. Add 100 (101, 109...) to also ignore manually hidden rows.

3. Lookups

VLOOKUP

```

=VLOOKUP(lookup_value, table_array, col_index, [range_lookup])
=VLOOKUP($A2, Products!$A$2:$F$60, 3, FALSE)

```

- **Always pass FALSE** (exact match). `TRUE` assumes the table is sorted ascending and silently returns approximate matches.
- Lookup column must be the **leftmost** column of `table_array`.
- `col_index` is a hard-coded number — insert a column in the source and the formula silently returns the wrong field. This is the main argument against VLOOKUP.

INDEX + MATCH (preferred)

```

=INDEX(return_range, MATCH(lookup_value, lookup_range, 0))
=INDEX(Products!$C:$C, MATCH($A2, Products!$A:$A, 0))

```

- `MATCH` third argument: `0` = exact (use this), `1` = largest $\leq$ (needs ascending), `-1` = smallest $\geq$ (needs descending).
- Works right-to-left, survives column insertion, and reads only the two columns it needs.

Two-way lookup (row and column):

```

=INDEX(data, MATCH(row_val, row_hdrs, 0), MATCH(col_val, col_hdrs, 0))

```

Multi-criteria lookup without array formulas:

```

=INDEX(C:C, MATCH(1, (A:A=E1)*(B:B=E2), 0)) ' Ctrl+Shift+Enter in older Excel
=SUMPRODUCT((A2:A100=E1)*(B2:B100=E2)*C2:C100) ' if the result is numeric

```

XLOOKUP (Excel 365 / 2021+)

```

=XLOOKUP(lookup, lookup_array, return_array, [if_not_found], [match_mode], [search_mode])
=XLOOKUP($A2, Products!$A:$A, Products!$C:$C, "Not found")

```

Exact match is the DEFAULT. Built-in not-found handling. Can return a whole row/column. `search_mode = -1` searches bottom-up (useful for “most recent” lookups).

Assessment tip: if the machine has Excel 365, use XLOOKUP and say why. If you're unsure of the version, INDEX/MATCH always works. Never lead with VLOOKUP unless asked for it specifically.

4. Data Cleaning Workflow

Before any analysis, spend two minutes on reconnaissance:

1. `Ctrl+End` — find the true extent of the data. Reveals stray cells far below the visible range.
2. `Ctrl+Shift+L` — turn on filters; open each key column's dropdown to see actual distinct values. Casing and whitespace variants show up

- immediately.
3. Check for blanks: `=COUNTBLANK(range)` , or filter to Blanks.
4. Check for duplicates: **Data > Remove Duplicates**, or flag with `=COUNTIF($A$2:$A$1000, A2)>1` .

Cleaning tools:

Task	Method
Trim whitespace	<code>=TRIM(A2)</code> then paste-special as values
Fix casing	<code>=PROPER(A2)</code> / <code>=UPPER(A2)</code>
Split a column	Data > Text to Columns (delimited or fixed width)
Merge columns	<code>=A2&" "&B2</code> or Flash Fill (<code>Ctrl+E</code>)
Remove duplicates	Data > Remove Duplicates (select which columns define a dupe)
Find/replace	<code>Ctrl+H</code> ; tick “Match entire cell contents” when needed
Convert text-numbers	<code>=VALUE(A2)</code> , or Text to Columns > Finish
Fill blanks	<code>Ctrl+G</code> > Special > Blanks, type value, <code>Ctrl+Enter</code>

Flash Fill (`Ctrl+E`) infers a pattern from one or two examples. Fast for splitting names or reformatting IDs, but it produces static values, not formulas — it won’t update when the source changes.

Data Validation (Data > Data Validation): restrict entry to a list, a number range, or a date range. The List source can be a range (`=H$2:H$5`) or a typed comma-separated list. This is how you build a dropdown for an interactive summary cell.

5. Pivot Tables

Creating one

`Alt + N + V` (or Insert > PivotTable). **Convert your source to a Table first (`Ctrl+T`)** — then the pivot’s source expands automatically when rows are added. Otherwise the source is a fixed range and new rows are silently excluded on refresh.

The four zones

Zone	What it does
Rows	One row per distinct value (nest multiple fields for hierarchy)
Columns	One column per distinct value — keep low-cardinality
Values	The aggregation (Sum, Count, Average, Max, Min, StdDev...)
Filters	Page-level filter above the pivot

Why “Count of Revenue” appears instead of “Sum”

Two causes, and this is a favourite exam question: 1. The column contains **blanks** or **text** anywhere in the range — Excel defaults to Count when the field isn’t purely numeric. 2. The numbers are stored **as text** (left-aligned, possibly with a green triangle).

Fix the data, refresh, then right-click the value field > Summarize Values By > Sum. Or double-click the field header to open Value Field Settings.

Show Values As

Right-click a value > **Show Values As**: - `% of Grand Total` , `% of Column Total` , `% of Row Total` - `% of Parent Row Total` — useful with nested row fields - `Difference From / % Difference From` — pick a base field and base item (e.g. previous month) - `Running Total In` - `Rank Largest to Smallest`

Grouping

- **Dates**: right-click any date in the pivot > Group > select Years, Quarters, Months (multi-select to nest). Excel creates the hierarchy automatically — you don’t need helper columns.
- **Numbers**: right-click > Group > set Starting at, Ending at, By. This is how you bucket ShipDays into 1-3, 4-6, 7-9, 10+.
- **Text**: select several row items > right-click > Group to make a manual grouping.

Calculated Field vs Calculated Item

Crucial distinction, frequently tested.

	Calculated Field	Calculated Item
Operates on	Whole fields (columns)	Individual items within one field
Appears as	A new field in the Values area	A new row/column within an existing field
Example	<code>Margin = Revenue - Cost</code>	<code>Q1+Q2</code> as a new item inside the Quarter field
Created via	PivotTable Analyze > Fields, Items & Sets > Calculated Field	...> Calculated Item (select an item in the field first)

The big gotcha: a Calculated Field computes on the **sum of the underlying data**, not on the sum of the displayed values. So `Margin% = (Revenue-Cost)/Revenue` gives `SUM(Revenue-Cost)/SUM(Revenue)` — usually what you want. But a field like `Avg Price = Revenue/Units` returns the ratio of totals, not the average of ratios. Be able to explain this.

Slicers and Timelines

- **Slicer:** PivotTable Analyze > Insert Slicer. Visual filter buttons. One slicer can control **multiple pivots** via Report Connections (right-click slicer > Report Connections) — essential for dashboards.
- **Timeline:** Insert Timeline. Date-only slicer with Years/Quarters/Months/Days granularity.

Other essentials

- **Refresh:** `Alt+F5` for one pivot, `Ctrl+Alt+F5` for all. Pivots do **not** auto-refresh when the source changes.
- **Show Details:** double-click any value cell — Excel generates a new sheet with the underlying rows. Great for auditing an unexpected number.
- **GETPIVOTDATA:** typing `=` and clicking a pivot cell auto-generates this. It's robust to the pivot's layout changing, unlike a plain cell reference.

```
=GETPIVOTDATA("Revenue", $A$3, "Region", "West", "Quarter", "2025-Q4")
```

To turn the auto-generation off: PivotTable Analyze > Options dropdown > untick Generate GetPivotData.

- **Value Filters:** right-click a row label > Filter > Top 10 (works for any N, and for Top N by any value field).
- **Repeat item labels:** Design > Report Layout > Repeat All Item Labels — needed when you want to use the pivot output as a flat data source.
- **Report Layout:** Compact (default), Outline, Tabular. Tabular puts each field in its own column — closest to a normal table.

6. Pivot Charts, Charts & Dashboards

Chart type selection

Question	Chart
Trend over time	Line (or Area for cumulative)
Compare categories	Column (vertical) or Bar (horizontal, for long labels)
Part-to-whole	Stacked column / 100% stacked; pie only for ≤5 slices
Relationship between two numerics	Scatter
Distribution	Histogram or Box & Whisker
Two different scales	Combo chart with a secondary axis
Progress to target	Bullet-style bar, or gauge

Building a dashboard

The Week 1 deliverable. A repeatable recipe:

1. **Separate your layers.** Sheet 1 = raw data (as a Table). Sheet 2 = pivots and helper calcs. Sheet 3 = the dashboard itself. Never build charts directly on raw data.
2. **Build each pivot** for one specific question. One pivot per chart.
3. **Insert pivot charts** (select pivot > Insert > chart type), then move them to the dashboard sheet (right-click chart > Move Chart > Object in Dashboard).
4. **Add KPI cards:** big single numbers in merged cells, driven by `GETPIVOTDATA` or `SUMIFS`. Format large, bold, with a coloured fill.
5. **Add slicers** and connect them to all pivots via Report Connections, so one click filters the whole dashboard.
6. **Clean up the chrome:** delete gridlines, remove legends where a single series makes them redundant, delete the chart border, set a consistent colour palette, add data labels instead of an axis where it reads better.
7. **Hide the working sheets** and turn off gridlines on the dashboard sheet (View > untick Gridlines).

Conditional formatting

Home > Conditional Formatting: - **Highlight Cells Rules** — greater than, between, text contains, duplicate values - **Top/Bottom Rules** — Top 10 items, Top 10%, above average - **Data Bars / Colour Scales / Icon Sets** — in-cell visuals, good for dense tables - **Use a formula** — the flexible one. Select the range starting at the active cell, then: `=D2>1000` ' highlight the ENTIRE ROW where column D exceeds 1000 `=AND($B2="West", $D2>500)` `=COUNTIF($A$2:$A$100, $A2)>1` ' highlight duplicates Note the mixed reference: `$D2` locks the column so the rule applies across the row, while the row number varies.

Sparklines

Insert > Sparklines > Line/Column/Win-Loss. A tiny in-cell chart per row. Good for a trend column beside a KPI table.

7. Power Query (Get & Transform)

Data > Get Data. The right tool when cleaning must be **repeatable** — the steps are recorded and re-run on refresh, unlike manual edits. Typical flow: Get Data > From File/Table > the Power Query Editor opens > apply steps > Close & Load.

Common transforms: Remove Columns, Remove Duplicates, Filter Rows, Transform > Format > Trim/Lowercase, Replace Values, Split Column, Change Type, Unpivot Columns (turns a wide cross-tab into long/tidy form — very handy), Merge Queries (a join), Append Queries (a union), Group By.

Every step appears in the Applied Steps pane and can be reordered or deleted. This is the audit trail that manual cleaning lacks.

8. Keyboard Shortcuts

Hands-on assessments are won on speed. Unplug the mouse while drilling.

Navigation & selection

Keys	Action
Ctrl+Arrow	Jump to the edge of the data block
Ctrl+Shift+Arrow	Select to the edge
Ctrl+End / Ctrl+Home	Last used cell / A1
Ctrl+A	Select the current region
Ctrl+Space / Shift+Space	Select column / row
Ctrl+PgUp/PgDn	Previous / next worksheet
F5 then Special	Go To Special (blanks, formulas, constants)

Editing

Keys	Action
F2	Edit in place
F4	Cycle absolute/relative (while editing); repeat last action (otherwise)
Ctrl+D / Ctrl+R	Fill down / fill right
Ctrl+Enter	Fill the entire selection with what you typed
Alt+Enter	Line break inside a cell
Ctrl+; / Ctrl+Shift+;	Insert today's date / current time
Ctrl+Shift+V	Paste Special
Ctrl+- / Ctrl+Shift++	Delete / insert cells

Formatting & features

Keys	Action
Ctrl+T	Create Table
Ctrl+Shift+L	Toggle filters
Alt+=	AutoSum
Alt+N+V	Insert PivotTable
Alt+H+O+I	Autofit column width
Ctrl+1	Format Cells dialog
Ctrl+Shift+\$ / % / #	Currency / percent / date format
Alt+F5	Refresh pivot
Ctrl+'` ``	Toggle show-formulas view

9. Excel Tables (Ctrl+T)

Converting a range to a Table changes several things at once: - **Structured references:** `=SUM(Sales[Revenue])` instead of `=SUM(D2:D5000)` . Readable and self-documenting. - **Auto-expansion:** new rows are included in every formula, chart and pivot that references the table. - Formulas **auto-fill** down the whole column when entered in one cell. - Built-in banded formatting, filter buttons, and an optional Total Row (`Ctrl+Shift+T`).

Structured reference syntax: `Table1[@Revenue]` = this row's value; `Table1[Revenue]` = the whole column; `Table1[#Headers]` , `Table1[#Totals]` , `Table1[#All]` .

Rename the table under Table Design > Table Name — `Sales` reads far better than `Table1` .

10. Errors and What They Mean

Error	Cause	Typical fix
#N/A	Lookup value not found	Check for whitespace/type mismatch; wrap in <code>IFNA</code>
#VALUE!	Wrong data type in an argument	Text where a number is expected
#REF!	Reference to a deleted cell	Rebuild the formula; usually unrecoverable
#DIV/0!	Division by zero or blank	<code>=IFERROR(a/b, 0)</code>
#NAME?	Misspelled function or undefined name	Check spelling; check the range name exists
#NUM!	Invalid numeric argument	e.g. <code>SQRT</code> of a negative
#NULL!	Intersection operator (space) with no overlap	Usually a missing comma between ranges
#SPILL!	Dynamic array blocked	Clear the cells in the spill range

Lookup returns #N/A but the value is visibly there — almost always one of: trailing space (fix with `TRIM`), number stored as text vs real number, or non-breaking space from a web/PDF paste (fix with `SUBSTITUTE(A2, CHAR(160), "")`).

11. Practice Checklist

Work these until each is under time. Use the practice workbook.

- ☐ Clean a dirty categorical column with `TRIM` / `PROPER` — 2 min
- ☐ Two-table lookup with `INDEX/MATCH` and `IFERROR` — 3 min
- ☐ Multi-criteria `SUMIFS` including a date-range condition — 3 min
- ☐ Build a quarter label from a date and look up a target — 4 min
- ☐ Pivot: Region × Quarter with date grouping and currency format — 5 min
- ☐ Same pivot as % of column total — 1 min
- ☐ Add a calculated field for margin % — 3 min
- ☐ Group a numeric field into buckets — 2 min
- ☐ Top 5 by value filter — 2 min
- ☐ Slicer + timeline connected to two pivots — 5 min
- ☐ Conditional format the top 10% of a column — 2 min
- ☐ Full dashboard: 3 charts + 3 KPI cards + slicers — 25 min

Python & Pandas — Complete Reference

EY Probation Training — Week 1. Roadmap scope: basics, data types, loops, functions, lists, dictionaries, pandas basics. Outcome bar: “write Python scripts and perform basic data analysis.”

1. Data Types

Type	Mutable	Ordered	Literal	Notes
<code>int</code> , <code>float</code> , <code>bool</code>	—	—	<code>5</code> , <code>2.5</code> , <code>True</code>	<code>bool</code> is a subclass of <code>int</code>
<code>str</code>	No	Yes	<code>"text"</code>	Immutable — methods return new strings
<code>list</code>	Yes	Yes	<code>[1, 2, 3]</code>	The default sequence
<code>tuple</code>	No	Yes	<code>(1, 2)</code>	Hashable, so usable as a dict key
<code>dict</code>	Yes	Yes (3.7+)	<code>{"a": 1}</code>	Keys must be hashable
<code>set</code>	Yes	No	<code>{1, 2}</code>	Unique elements, fast membership
<code>NoneType</code>	—	—	<code>None</code>	Test with <code>is None</code> , never <code>== None</code>

```
type(x)           # exact type
isinstance(x, int) # preferred check – respects inheritance
int("42"); float("3.14"); str(42); list("abc"); bool(0) # conversions
```

2. Strings

```
s = " Hello, World "
s.strip(); s.lstrip(); s.rstrip()
s.lower(); s.upper(); s.title()
s.replace("World", "Python")
s.split(",")           # -> [' Hello', ' World ']
", ".join(["a", "b", "c"]) # -> 'a,b,c'
s.startswith("H"); s.endswith("d"); "World" in s
s.find("World")         # -1 if absent
s.index("World")        # raises ValueError if absent
```

f-strings (use these, not `%` or `.format()`):

```
name, val = "Q3", 1234.5678
f"{name}: {val:,.2f}"      # 'Q3: 1,234.57'
f"{val:>12.1f}"            # right-align in 12 chars
f"{0.4567:.1%}"           # '45.7%'
f"{name=}"                 # 'name=Q3' (debug form)
```

Slicing (applies to lists too):

```
s[0]; s[-1]; s[2:5]; s[:3]; s[3:]; s[::2]; s[::-1] # last one reverses
```

3. Lists

```
xs = [3, 1, 4, 1, 5]
xs.append(9)           # add one at the end
xs.extend([2, 6])      # add many
xs.insert(0, 0)        # at an index
xs.remove(1)           # first matching VALUE
xs.pop()               # last item (and returns it)
xs.pop(0)              # by index
xs.sort()              # IN PLACE, returns None
sorted(xs)             # returns a NEW list
xs.sort(reverse=True)
xs.sort(key=len)       # sort by a computed key
xs.reverse(); xs.count(1); xs.index(4)
len(xs); sum(xs); min(xs); max(xs)
```

Common trap: `xs.sort()` returns `None`. Writing `xs = xs.sort()` destroys your list. Use `sorted()` when you want a return value.

List comprehensions

```
squares = [x**2 for x in range(10)]
evens   = [x for x in xs if x % 2 == 0]
labelled = [f"item_{x}" for x in xs]
pairs    = [(x, y) for x in [1,2] for y in "ab"]    # nested loops
cond     = [x if x > 0 else 0 for x in xs]         # ternary goes BEFORE 'for'
```

Filtering `if` goes after the `for`; a conditional *value* goes before it. Mixing these up is a classic error.

```
{k: v for k, v in pairs}    # dict comprehension
{x % 3 for x in range(10)} # set comprehension
(x**2 for x in range(10))  # GENERATOR – lazy, not a list
```

Copying

```
a = [1, 2, 3]
b = a          # SAME object – mutating b changes a
c = a.copy()   # shallow copy (or a[:], or list(a))
import copy; d = copy.deepcopy(nested) # for nested structures
```

4. Dictionaries

```
d = {"a": 1, "b": 2}
d["a"]          # KeyError if missing
d.get("z")       # None if missing
d.get("z", 0)    # default if missing
d["c"] = 3
d.setdefault("e", []).append(1)
d.update({"f": 6})
d.pop("a"); del d["b"]
"a" in d         # membership tests KEYS

d.keys(); d.values(); d.items()
for k, v in d.items(): print(k, v)

sorted(d.items(), key=lambda kv: kv[1], reverse=True) # sort by value
max(d, key=d.get)                                   # key with the largest value
```

Counting pattern:

```
counts = {}
for w in words:
    counts[w] = counts.get(w, 0) + 1

from collections import Counter, defaultdict
Counter(words).most_common(3)
groups = defaultdict(list)
for name, dept in rows: groups[dept].append(name)
```

5. Control Flow


```

if x > 10:      pass
elif x > 5:     pass
else:          pass

for i in range(5):      # 0..4
for i in range(2, 10, 2): # 2,4,6,8
for item in xs:
for i, item in enumerate(xs, start=1):
for a, b in zip(xs, ys):
for k, v in d.items():

while cond:
    ...
    if done: break
    if skip: continue
else:
    ... # runs only if the loop was NOT broken out of

```

`enumerate` and `zip` are what assessments look for instead of `for i in range(len(xs))`.

6. Functions

```

def summarise(data, top=5, *args, label="Report", **kwargs):
    """One-line summary.

    Args:
        data: the input list.
        top: how many rows to return.

    Returns:
        list: the top-n items.
    """
    return sorted(data, reverse=True)[:top]

```

- Positional args first, then defaults, then `*args`, then keyword-only, then `**kwargs`.
- **Never use a mutable default.** `def f(x, acc=[])` shares one list across all calls. Use `acc=None` then `if acc is None: acc = []`.
- Type hints are good practice: `def f(x: int, names: list[str]) -> dict:`

```

sq = lambda x: x**2 # anonymous function
sorted(rows, key=lambda r: (r["dept"], -r["pay"])) # multi-key sort
list(map(str.upper, names))
list(filter(lambda x: x > 0, xs))

```

Comprehensions are preferred over `map / filter` in modern Python style.

7. Files and Errors

```

with open("data.txt") as f:      # 'with' closes the file automatically
    text = f.read()
    # f.readlines() / for line in f:

with open("out.txt", "w") as f:
    f.write("hello\n")
    f.writelines(lines)

import csv
with open("data.csv", newline="") as f:
    for row in csv.DictReader(f):
        print(row["Region"])

import json
cfg = json.load(open("cfg.json"))
json.dump(obj, open("out.json", "w"), indent=2)

```

Modes: `r` read, `w` write (truncates), `a` append, `rb / wb` binary.

```

try:
    val = int(s)
except ValueError as e:
    print(f"bad value: {e}")
except (TypeError, KeyError):
    pass
else:
    print("no exception")    # runs if try succeeded
finally:
    cleanup()                # always runs
raise ValueError("explain what went wrong")

```

8. Pandas — Creating and Inspecting

```

import pandas as pd, numpy as np

df = pd.read_csv("file.csv")
df = pd.read_csv("f.csv", parse_dates=["date"], dtype={"id": str},
                usecols=["a", "b"], na_values=["NA", "-"], thousands=",")
df = pd.read_excel("f.xlsx", sheet_name="Orders")
df = pd.DataFrame({"a": [1,2], "b": ["x", "y"]})

df.head(); df.tail(); df.sample(5)
df.shape; df.columns; df.dtypes; df.index
df.info()          # dtypes + non-null counts + memory
df.describe()      # numeric summary
df.describe(include="object") # categorical summary
df.nunique(); df["col"].unique(); df["col"].value_counts()
df.isna().sum()    # nulls per column
df.memory_usage(deep=True)

```

9. Pandas — Selecting

```

df["col"]          # Series
df[["a", "b"]]     # DataFrame

# .loc = LABEL based (end-inclusive) | .iloc = POSITION based (end-exclusive)
df.loc[0, "col"]
df.loc[0:5, "a":"c"] # includes row 5 AND column c
df.iloc[0:5, 0:3]    # excludes row 5 and column 3
df.loc[df["qty"] > 5, ["region", "qty"]]

# Boolean filtering – each condition needs parentheses
df[(df.qty > 5) & (df.region == "West")] # & | ~ NOT and/or/not
df[df.region.isin(["West", "East"])]
df[df.region.str.contains("west", case=False, na=False)]
df[df.qty.between(3, 8)]
df[df.col.isna()] / df[df.col.notna()]
df.query("qty > 5 and region == 'West'") # readable alternative

```

.loc is end-inclusive, **.iloc** is end-exclusive. Guaranteed to appear in an assessment.

10. Pandas — Cleaning

```

df = df.rename(columns={"old": "new"})
df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")

df.drop(columns=["a", "b"])
df.drop_duplicates() # all columns
df.drop_duplicates(subset=["id"], keep="last")
df.duplicated(subset=["id"]).sum()

df.dropna() # any null in any column
df.dropna(subset=["qty"])
df.dropna(thresh=3) # keep rows with >=3 non-nulls
df.fillna(0)
df.fillna({"qty": 0, "region": "Unknown"})
df["qty"].fillna(df["qty"].median())
df["v"].ffill() / df["v"].bfill() # forward / backward fill

df["qty"] = df["qty"].astype(int)
df["date"] = pd.to_datetime(df["date"], errors="coerce") # bad -> NaT
df["amt"] = pd.to_numeric(df["amt"], errors="coerce")

df["region"] = df["region"].str.strip().str.title()
df["region"] = df["region"].replace({"W": "West"})
df["band"] = df["qty"].map({1: "low", 2: "mid"})
df["flag"] = np.where(df.qty > 5, "high", "low")
df["band"] = pd.cut(df.qty, bins=[0,3,6,99], labels=["low","mid","high"])
df["decile"] = pd.qcut(df.amt, 10, labels=False) # equal-COUNT bins

```

`pd.cut` = equal-width bins you specify. `pd.qcut` = equal-frequency (quantile) bins. Know the difference.

11. Pandas — Aggregation

```

df.groupby("region")["revenue"].sum()
df.groupby(["region", "channel"])["revenue"].agg(["sum", "mean", "count"])

df.groupby("region").agg(
    revenue=("revenue", "sum"),
    orders=("order_id", "count"),
    avg_qty=("qty", "mean"),
    n_products=("sku", "nunique"),
).reset_index() # named aggregation – preferred

df.groupby("region")["revenue"].transform("sum") # broadcasts back to row level
df.groupby("region").apply(lambda g: g.nlargest(3, "revenue"))
df.groupby("region").filter(lambda g: len(g) > 100)

```

`agg` reduces to one row per group; `transform` returns the original shape (use it to compute a share-of-group column); `filter` keeps or drops whole groups.

```

df.pivot_table(index="region", columns="quarter", values="revenue",
               aggfunc="sum", margins=True, fill_value=0)
pd.crosstab(df.region, df.channel, normalize="index")
df.melt(id_vars=["region"], var_name="metric", value_name="value") # wide -> long

```

`pivot_table` is the direct pandas analogue of an Excel pivot — same mental model.

12. Pandas — Joining & Combining

```

pd.merge(left, right, on="key", how="inner") # inner|left|right|outer|cross
pd.merge(l, r, left_on="id", right_on="ref", how="left")
pd.merge(l, r, on="key", suffixes=("_l", "_r"))
pd.merge(l, r, on="key", how="left", indicator=True) # adds _merge column
l.join(r, on="key") # index-based
pd.concat([df1, df2], axis=0, ignore_index=True) # stack rows
pd.concat([df1, df2], axis=1) # side by side

```

Always check row counts before and after a merge. If the count grew, the right-hand key isn't unique and you have fan-out — every downstream sum is now inflated. `indicator=True` plus `df._merge.value_counts()` tells you exactly how many rows matched.

13. Pandas — Sorting, Ranking, Windows

```

df.sort_values("revenue", ascending=False)
df.sort_values(["region", "revenue"], ascending=[True, False])
df.nlargest(5, "revenue"); df.nsmallest(5, "revenue")

df["rank"] = df.groupby("region")["revenue"].rank(method="dense", ascending=False)
# method: average (default), min, max, first, dense

df["cum"] = df.groupby("region")["revenue"].cumsum()
df["ma3"] = df["revenue"].rolling(3).mean()
df["ma3c"] = df["revenue"].rolling(3, center=True, min_periods=1).mean()
df["ewm"] = df["revenue"].ewm(span=3).mean()
df["lag1"] = df.groupby("sku")["revenue"].shift(1)
df["pct"] = df["revenue"].pct_change()
df["expand"] = df["revenue"].expanding().mean()

```

14. Pandas — Dates

```

df["date"] = pd.to_datetime(df["date"])
df["year"] = df.date.dt.year
df["month"] = df.date.dt.month
df["quarter"] = df.date.dt.quarter
df["dow"] = df.date.dt.dayofweek # Monday = 0
df["dayname"] = df.date.dt.day_name()
df["week"] = df.date.dt.isocalendar().week
df["month_start"] = df.date.dt.to_period("M").dt.to_timestamp()

df.set_index("date").resample("M")["revenue"].sum() # M, W, Q, D, H
df.set_index("date").resample("W").agg({"revenue": "sum", "qty": "mean"})
pd.date_range("2025-01-01", periods=12, freq="MS")
df.date.max() - df.date.min() # Timedelta

```

`.dt` is the accessor for datetime methods on a Series, just as `.str` is for strings.

15. Output

```

df.to_csv("out.csv", index=False)
df.to_excel("out.xlsx", sheet_name="Summary", index=False)
with pd.ExcelWriter("multi.xlsx") as w:
    df1.to_excel(w, sheet_name="Raw", index=False)
    df2.to_excel(w, sheet_name="Summary", index=False)
df.to_parquet("out.parquet")

```

`index=False` is almost always what you want for CSV — otherwise you get a stray unnamed first column that causes trouble on re-read.

16. Gotchas Worth Memorising

SettingWithCopyWarning. Chained indexing may operate on a copy, so the assignment silently does nothing:

```

df[df.qty > 5]["flag"] = 1 # WRONG — may not stick
df.loc[df.qty > 5, "flag"] = 1 # RIGHT
sub = df[df.qty > 5].copy() # explicit copy if you'll modify it

```

`inplace=True` is not faster and is being discouraged. Prefer reassignment: `df = df.dropna()`.

`&`, `|`, `~` for element-wise boolean ops — `and` / `or` / `not` raise “truth value of a Series is ambiguous”.

Parentheses are mandatory around each condition because `&` binds tighter than `>`.

`NaN != NaN`. Use `.isna()`, never `== np.nan`.

`axis=0` operates down rows (per column); `axis=1` operates across columns (per row). `df.mean()` defaults to `axis=0` — a per-column mean.

`apply` is a loop. Prefer vectorised operations, `np.where`, or `.map` for a dict lookup. `apply(axis=1)` in particular is very slow on large frames.

17. Practice Checklist

- ☐ Read a messy CSV with the right `dtype`, `parse_dates`, `na_values`
- ☐ Profile it: shape, dtypes, nulls per column, distinct counts
- ☐ Clean: trim/case-normalise strings, coerce types, handle nulls with a justified strategy
- ☐ Dedup on a business key, not on all columns
- ☐ Derive columns: date parts, a computed metric, a binned category

- ☐ `groupby().agg()` with named aggregations returning several metrics
- ☐ `pivot_table` cross-tab with margins
- ☐ Merge two frames and verify the row count is unchanged
- ☐ Rank within group, and compute share-of-group with `transform`
- ☐ Rolling mean and lag features on a time-ordered frame
- ☐ Resample daily data to monthly
- ☐ Write a multi-sheet Excel output

PySpark — Complete Reference

EY Probation Training — Week 1 (basics, DataFrames, transformations, aggregations, joins) and Week 4 (Databricks). Roadmap outcome bar is “understand and write basic PySpark code” — this goes further, because PySpark is also your professional differentiator.

1. Architecture in One Page

Driver runs your code, builds the DAG, and schedules work. **Executors** run tasks and hold cached data. The **Cluster Manager** (YARN, Kubernetes, Standalone, Databricks) allocates executors.

- **Job** — one action triggers one job.
- **Stage** — a job is split at shuffle boundaries.
- **Task** — one unit of work per partition within a stage.

Catalyst is the query optimiser: parsed plan → analysed plan → optimised logical plan → physical plan. **Tungsten** handles off-heap memory management and whole-stage code generation.

Lazy evaluation: transformations only build the DAG. Nothing executes until an action.

Transformations (lazy)	Actions (trigger execution)
<code>select</code> , <code>filter</code> , <code>withColumn</code> , <code>drop</code>	<code>count</code> , <code>collect</code> , <code>show</code> , <code>first</code> , <code>take</code>
<code>groupBy</code> , <code>agg</code> , <code>join</code> , <code>orderBy</code>	<code>write</code> , <code>foreach</code> , <code>toPandas</code>
<code>distinct</code> , <code>repartition</code> , <code>union</code>	<code>cache</code> is lazy; <code>.count()</code> materialises it

Narrow (no shuffle)	Wide (shuffle)
<code>select</code> , <code>filter</code> , <code>withColumn</code> , <code>map</code> , <code>union</code> , <code>coalesce</code>	<code>groupBy</code> , <code>join</code> , <code>distinct</code> , <code>orderBy</code> , <code>repartition</code> , window functions

2. Session and Reading

```
from pyspark.sql import SparkSession, functions as F, Window
from pyspark.sql.types import *

spark = (SparkSession.builder
        .appName("app")
        .master("local[*]")
        .config("spark.sql.shuffle.partitions", "8")
        .getOrCreate())
spark.sparkContext.setLogLevel("WARN")
```

```
# Explicit schema – one pass, deterministic. Preferred in pipelines.
schema = StructType([
    StructField("order_id", StringType(), True),
    StructField("order_date", DateType(), True),
    StructField("qty", IntegerType(), True),
    StructField("amount", DoubleType(), True),
])
df = spark.read.csv("f.csv", header=True, schema=schema)

# inferSchema costs an EXTRA full pass over the data
df = spark.read.csv("f.csv", header=True, inferSchema=True)

df = spark.read.option("escape", "'').csv(...) # when text fields contain quotes
df = spark.read.parquet("path/")
df = spark.read.json("path/")
df = spark.read.format("delta").load("path/")
```

CSV quoting trap: RFC-4180 escapes an inner quote by doubling it (""), but Spark’s CSV reader defaults to backslash. If a text column contains quotes, pass `escape='\"'` or fields silently misalign — and the corruption surfaces later as a baffling cast error.

3. Inspecting

```
df.show(20, truncate=False); df.printSchema(); df.columns; df.dtypes
df.count(); df.describe().show(); df.summary().show()
df.explain(); df.explain(True) # True = all four plans
df.rdd.getNumPartitions()
df.select([F.sum(F.col(c).isNull().cast("int")).alias(c) for c in df.columns]).show()
```

That last line gives null counts for every column in a single pass — much better than one action per column.

4. Columns and Expressions

```
df.select("a", "b")
df.select(F.col("a"), F.col("b").alias("bb"))
df.select(F.expr("qty * amount as revenue"))
df.selectExpr("qty * amount as revenue", "upper(region) as r")

df.withColumn("revenue", F.col("qty") * F.col("amount"))
df.withColumnRenamed("old", "new")
df.drop("a", "b")
df.filter(F.col("qty") > 5)
df.where("qty > 5 AND region = 'West'")      # SQL string form
df.distinct(); df.dropDuplicates(["order_id"])
df.orderBy(F.desc("revenue"), F.asc("region"))
df.limit(10)
```

Conditional logic — the `when / otherwise` chain, evaluated top-down, first match wins:

```
F.when(F.col("amt") < 1000, "Low") \
  .when(F.col("amt") < 4000, "Medium") \
  .otherwise("High")
```

Common functions:

```
F.lit(0); F.coalesce(a, b, c); F.greatest(a,b); F.least(a,b)
F.round(x, 2); F.abs(x); F.floor(x); F.ceil(x); F.sqrt(x)
F.upper/lower/initcap/trim/ltrim/rtrim(col)
F.concat_ws("-", a, b); F.substring(c, 1, 5); F.length(c)
F.split(c, ","); F.regexp_replace(c, r"\s+", " "); F.regexp_extract(c, r"(\d+)", 1)
F.year/month/dayofmonth/quarter/hour/weekofyear(d)
F.date_format(d, "yyyy-MM"); F.datediff(a,b); F.add_months(d,1)
F.to_date(c, "yyyy-MM-dd"); F.to_timestamp(c, fmt); F.trunc(d, "month")
F.cast / col.cast("double")
```

Null handling:

```
df.na.drop(); df.na.drop(subset=["qty"]); df.na.drop(how="all")
df.na.fill(0); df.na.fill({"qty": 0, "region": "Unknown"})
df.na.replace(["W"], ["West"], "region")
F.col("x").isNull() / .isNotNull()
```

Spark 4.x ANSI mode is ON by default. A failed cast or timestamp parse now RAISES instead of returning null. Use `try_cast`, `try_to_timestamp` for version-independent null-on-failure behaviour. In Spark 3.x the same code silently produced nulls. Know which version you're on.

5. Aggregations

```
df.groupBy("region").count()
df.groupBy("region").agg(
  F.sum("revenue").alias("revenue"),
  F.avg("qty").alias("avg_qty"),
  F.countDistinct("sku").alias("skus"),
  F.approx_count_distinct("customer_id", 0.05).alias("approx_customers"),
  F.min("date").alias("first"), F.max("date").alias("last"),
  F.stddev("amount"), F.expr("percentile_approx(amount, 0.5)").alias("median"),
  F.collect_list("sku"), F.collect_set("sku"),
)
df.agg(F.sum("revenue")).first()[0]      # scalar out

df.groupBy("region").pivot("quarter").agg(F.sum("revenue"))      # Excel pivot equivalent
df.groupBy("region").pivot("quarter", ["Q1", "Q2"]).sum("revenue") # faster: values listed
df.rollup("region", "city").agg(F.sum("revenue"))      # subtotals
df.cube("region", "channel").agg(F.sum("revenue"))      # all combinations
```

`countDistinct` is exact but requires a shuffle; `approx_count_distinct` uses HyperLogLog and is far cheaper at scale when a small error is acceptable.

6. Joins

```
a.join(b, "key", "inner")           # string form: ONE merged key column
a.join(b, ["k1", "k2"], "left")    # multi-key
a.join(b, a.id == b.ref, "left")    # expression form: BOTH columns kept
a.join(F.broadcast(b), "key", "inner") # force broadcast
```

Type	Keeps	Notes
inner	Matching rows only	Default
left	All left + matched right	Right columns null when unmatched
right	All right + matched left	
outer / full	Everything	
left_semi	Left rows that HAVE a match	Left columns only; cannot fan out
left_anti	Left rows with NO match	Left columns only; the integrity-check tool
cross	Cartesian product	Explicit only; guard carefully

Before every join, check two things:

```
dup = dim.groupBy("key").count().filter("count > 1").count() # is the key unique?
orphan = fact.join(dim.select("key").distinct(), "key", "left_anti").count()
assert dup == 0, "dimension key not unique -- join will fan out"
```

Join fan-out is the most damaging silent bug in Spark work. If the dimension key has duplicates, rows multiply and every downstream sum is overstated — no error, no warning. Assert row-count preservation on any join you expect to be 1:1.

Column collision: if both sides have a same-named non-key column, you get two columns and an `AMBIGUOUS_REFERENCE` error on any later reference. Drop or rename before joining.

Broadcast join: ships the small table to every executor, eliminating the shuffle. Automatic below `spark.sql.autoBroadcastJoinThreshold` (default 10 MB). The explicit hint matters when Spark's size estimate is wrong — after a filter, or with stale statistics. Look for `BroadcastHashJoin` vs `SortMergeJoin` in `.explain()`.

Range join (SCD type-2 / validity windows):

```
txn.join(prices, (txn.sku == prices.sku) &
              (txn.date >= prices.valid_from) &
              (txn.date <= prices.valid_to), "left")
```

Keep at least one equality in the condition. With no equi-key Spark falls back to `BroadcastNestedLoopJoin`, which is $O(n \times m)$.

7. Window Functions

```
w = Window.partitionBy("region").orderBy(F.desc("revenue"))

F.row_number().over(w)      # 1,2,3 – ties broken arbitrarily
F.rank().over(w)            # 1,2,2,4 – ties share, then SKIP
F.dense_rank().over(w)      # 1,2,2,3 – ties share, no skip
F.percent_rank().over(w)    # (rank-1)/(n-1)
F.ntile(4).over(w)          # quartile buckets
F.lag("x", 1).over(w); F.lead("x", 1).over(w)
F.first("x").over(w); F.last("x", ignorenulls=True).over(w)
```

Frames — the most-tested distinction

```
w.rowsBetween(-2, 0)           # 3 ROWS: previous 2 + current
w.rangeBetween(-1800, 0)       # 30 MINUTES of the ordering VALUE
w.rowsBetween(Window.unboundedPreceding, Window.currentRow) # cumulative
w.rowsBetween(Window.unboundedPreceding, Window.unboundedFollowing) # whole partition
w.rowsBetween(-3, 3)           # centred
```

- `rowsBetween` counts **rows**. `rangeBetween` counts **values of the ordering column** (needs a numeric/timestamp ordering key).
- With gaps in a time series these give different answers: 6 rows might span 50 minutes; 30 minutes might cover 3 rows.

The last_value trap: the DEFAULT frame for an ordered window is `rangeBetween(unboundedPreceding, currentRow)`. So `F.last("x").over(w)` returns the CURRENT row, not the final one. You must specify `unboundedFollowing` explicitly. `first()` happens to work by accident, which is why this bug survives review.

Common patterns:


```

# top N per group
df.withColumn("rn", F.row_number().over(w)).filter("rn <= 3")

# deduplicate deterministically
df.withColumn("rn", F.row_number().over(
    Window.partitionBy("id").orderBy(F.desc("updated_at")))).filter("rn = 1")

# running total
F.sum("rev").over(Window.partitionBy("r").orderBy("d")
    .rowsBetween(Window.unboundedPreceding, Window.currentRow))

# share of group total
F.col("rev") / F.sum("rev").over(Window.partitionBy("region"))

# sessionization: flag -> lag -> cumulative sum
(df.withColumn("is_f", (F.col("status")=="FAULT").cast("int"))
    .withColumn("prev", F.lag("is_f",1,0).over(w2))
    .withColumn("start", ((F.col("is_f")==1)&(F.col("prev")==0)).cast("int"))
    .withColumn("run_id", F.sum("start").over(w2.rowsBetween(
        Window.unboundedPreceding, Window.currentRow))))

```

8. Performance

Partitioning

```

df.rdd.getNumPartitions()
df.repartition(20)                # full shuffle, even sizes
df.repartition("store_id")        # hash by key – skewed keys land in one partition
df.coalesce(4)                   # merges partitions, NO shuffle, can only REDUCE
df.withColumn("pid", F.spark_partition_id()).groupBy("pid").count()

```

`coalesce` can only decrease the count because creating new partitions requires redistributing rows — which is a shuffle. Use `coalesce` before a write to reduce output files; `repartition` when you need balance.

`spark.sql.shuffle.partitions` defaults to 200 regardless of data size. Rule of thumb: 2-3× total cores, or target ~128 MB per partition. AQE's `coalescePartitions` now adjusts this at runtime.

Skew

Symptom: one task runs far longer than the rest — “99% done for 20 minutes”.

Salting — two-stage aggregation:

```

SALT = 16
(df.withColumn("salt", (F.rand()*SALT).cast("int"))
    .groupBy("key", "salt").agg(F.sum("v").alias("part"))
    .groupBy("key").agg(F.sum("part").alias("total")))

```

AQE handles skewed *joins* automatically by splitting the oversized partition:

```

spark.sql.adaptive.enabled                (true by default in 3.2+)
spark.sql.adaptive.skewJoin.enabled
spark.sql.adaptive.skewJoin.skewedPartitionFactor
spark.sql.adaptive.coalescePartitions.enabled

```

AQE does **not** fix a skewed `groupBy` on one hot key — salting is still your job.

Caching

```

df.cache()                             # = persist(MEMORY_AND_DISK); LAZY
df.persist(StorageLevel.MEMORY_ONLY)
df.count()                             # materialise it
df.unpersist()

```

When caching hurts: used only once; larger than memory (MEMORY_ONLY silently recomputes evicted partitions); starves execution memory; and it acts as an optimisation **barrier** — Catalyst can't push new filters through a cached DataFrame.

Other

- Prefer Parquet over CSV: columnar, compressed, carries schema, per-rowgroup min/max stats enable block skipping.
- Avoid `collect()` — it pulls everything to the driver. `spark.driver.maxResultSize` guards against OOM. Use `take(n)`, `show()`, or write to storage.
- Filter early, select only needed columns — though Catalyst usually does this for you unless a UDF blocks it.

9. Writing

```
df.write.mode("overwrite").parquet("path/")
df.write.mode("append").partitionBy("year", "month").parquet("path/")
df.write.format("delta").mode("overwrite").save("path/")
df.write.bucketBy(8, "customer_id").saveAsTable("tbl")
df.coalesce(1).write.mode("overwrite").csv("path/", header=True)
```

Mode	Behaviour
<code>overwrite</code>	Replaces the path
<code>append</code>	Adds files; watch for duplicates on retry
<code>ignore</code>	Silent no-op if the path exists — dangerous, hides failures
<code>errorifexists</code>	Default; throws

`partitionBy` creates directories (`channel=web/`) and enables **partition pruning** — look for `PartitionFilters` in the plan. Partition on a LOW-cardinality column you actually filter on. Partitioning by date at day-grain on a small table creates the **small-files problem**: hundreds of tiny files where metadata and task-launch overhead dominate the actual read.

`bucketBy` hashes rows into a fixed number of files by key. Two tables bucketed identically on the same key join with **no shuffle**. Requires `saveAsTable` .

10. Spark SQL

```
df.createOrReplaceTempView("orders")           # this session only
df.createOrReplaceGlobalTempView("orders")      # all sessions; global_temp.orders
df.write.saveAsTable("orders")                  # persisted to the metastore

spark.sql("""
WITH q AS (
  SELECT channel, YEAR(order_date) yr, SUM(qty*amount) rev
  FROM orders GROUP BY channel, YEAR(order_date)
)
SELECT *, ROUND(100.0*(rev - LAG(rev) OVER (PARTITION BY channel ORDER BY yr))
               / LAG(rev) OVER (PARTITION BY channel ORDER BY yr), 1) AS yoy
FROM q ORDER BY channel, yr
""").show()
```

SQL and the DataFrame API compile to the **same** Catalyst plan — there is no performance difference. Choose for maintainability: SQL is portable and analyst-readable; the DataFrame API composes into functions and unit-tests cleanly.

11. UDFs — Last Resort

```
# 1. Plain Python UDF — SLOWEST
@F.udf(StringType())
def bucket(a): return "Low" if a < 1000 else "High"

# 2. pandas UDF — vectorised via Arrow
from pyspark.sql.functions import pandas_udf
@pandas_udf(StringType())
def bucket_pd(s: pd.Series) -> pd.Series:
    return pd.cut(s, [-1,1000,1e9], labels=["Low","High"]).astype(str)

# 3. Native expression — FASTEST
F.when(F.col("amt") < 1000, "Low").otherwise("High")
```

Why the ordering: a Python UDF serialises every row to a Python process and back, one at a time, and is a black box to Catalyst — no pushdown, no whole-stage codegen. A pandas UDF moves rows in Arrow batches with a columnar zero-copy layout, amortising the crossing. A native expression never leaves the JVM and gets folded into generated bytecode.

Native > pandas_udf > python udf. Reach for a UDF only when no built-in expresses the logic.

12. Delta Lake (Databricks context)

```
df.write.format("delta").mode("overwrite").save(path)
spark.read.format("delta").option("versionAsOf", 3).load(path)    # time travel

from delta.tables import DeltaTable
dt = DeltaTable.forPath(spark, path)
dt.history().show()
dt.vacuum(168)                                                    # clean files older than 7 days
(dt.alias("t").merge(updates.alias("s"), "t.id = s.id")
 .whenMatchedUpdateAll().whenNotMatchedInsertAll()).execute()    # UPSERT
spark.sql("OPTIMIZE tbl ZORDER BY (customer_id)")
```

Delta adds ACID transactions, schema enforcement and evolution, time travel, and **MERGE** (upsert) on top of Parquet. **OPTIMIZE** compacts small files; **ZORDER** co-locates related data to improve skipping.

13. Practice Checklist

- ☐ Read with an explicit schema; explain why over **inferSchema**
- ☐ Null counts for all columns in one statement
- ☐ Clean dirty categoricals with **trim** / **initcap**
- ☐ Deduplicate with **row_number** and justify the ordering key
- ☐ Pre-join integrity checks: key uniqueness + **left_anti** orphan count
- ☐ All six join types with predicted row counts
- ☐ Force and verify a broadcast join in **.explain()**
- ☐ Multi-metric **agg()** plus a **pivot()**
- ☐ **row_number** / **rank** / **dense_rank** and explain the tie behaviour
- ☐ **rowsBetween** vs **rangeBetween** on data with gaps
- ☐ Detect skew, then fix it with salting; verify totals match
- ☐ Cache, time two actions, unpersist
- ☐ Write partitioned Parquet, confirm pruning in the plan
- ☐ Same query in SQL and DataFrame API; compare plans

SQL — Complete Reference

EY Probation Training — Week 2. Scope: fundamentals, joins, aggregations, subqueries, window functions, CTEs. Dataset: AdventureWorks.

This is the highest bar in the whole roadmap. Every other outcome says “understand” or “gain familiarity”; this one says **“able to independently write and optimize SQL queries.”** Prioritise accordingly.

1. Logical Execution Order

Not the order you write it. This explains most SQL confusion.

1. FROM / JOIN -> build the working set
2. WHERE -> filter individual ROWS
3. GROUP BY -> collapse into groups
4. HAVING -> filter GROUPS
5. SELECT -> compute output columns
6. DISTINCT
7. ORDER BY -> sort
8. LIMIT / TOP -> truncate

Consequences you must be able to state: - **You cannot use a `SELECT` alias in `WHERE`** — `WHERE` runs before `SELECT`. (Most engines DO allow it in `ORDER BY`, which runs after.) - **`WHERE` filters rows, `HAVING` filters groups.** `WHERE` cannot contain an aggregate; `HAVING` normally does. - **Window functions cannot appear in `WHERE` or `HAVING`** — they’re computed in step 5, after both. Wrap in a CTE or subquery first. This is a favourite exam question.

2. Basics

```
SELECT DISTINCT c.FirstName, c.LastName
FROM Person.Person AS c
WHERE c.LastName LIKE 'S%' AND c.EmailPromotion > 0
ORDER BY c.LastName ASC, c.FirstName DESC
OFFSET 0 ROWS FETCH NEXT 10 ROWS ONLY; -- ANSI; SQL Server also has TOP 10
```

Operators: `=` `<` `< >` `<=` `>=`, `BETWEEN a AND b` (inclusive), `IN (...)`, `LIKE`, `IS NULL`, `IS NOT NULL`, `AND` / `OR` / `NOT`.

Wildcards: `%` = any number of characters, `_` = exactly one.

NULL semantics — heavily tested

- `NULL = NULL` is **not true**; it’s unknown. Always use `IS NULL`.
- Any arithmetic with NULL yields NULL: `100 + NULL = NULL`.
- `NOT IN (SELECT ...)` returns **no rows** if the subquery contains a single NULL. Use `NOT EXISTS` instead — this is a real production bug, not just trivia.
- `COUNT(*)` counts rows; `COUNT(col)` **skips NULLs**. `COUNT(DISTINCT col)` also skips them.
- `AVG(col)` divides by the count of non-NULL values, not by the row count.

```
COALESCE(a, b, c) -- first non-null, ANSI standard
ISNULL(a, 0) -- SQL Server, two args only
NULLIF(a, b) -- NULL if a = b; classic divide-by-zero guard:
SELECT total / NULLIF(count, 0) FROM t;
```

CASE

```
SELECT OrderID,
CASE WHEN Total >= 10000 THEN 'Large'
     WHEN Total >= 1000 THEN 'Medium'
     ELSE 'Small' END AS Band,
CASE Status WHEN 1 THEN 'New' WHEN 5 THEN 'Shipped' ELSE 'Other' END AS StatusName
FROM Sales.SalesOrderHeader;
```

Evaluated top-down, first match wins. `ELSE` omitted means unmatched rows get NULL.

Conditional aggregation — the pivot idiom, worth memorising:

```
SELECT TerritoryID,
SUM(CASE WHEN YEAR(OrderDate) = 2013 THEN TotalDue ELSE 0 END) AS Rev2013,
SUM(CASE WHEN YEAR(OrderDate) = 2014 THEN TotalDue ELSE 0 END) AS Rev2014,
COUNT(CASE WHEN OnlineOrderFlag = 1 THEN 1 END) AS OnlineOrders
FROM Sales.SalesOrderHeader GROUP BY TerritoryID;
```

Note `COUNT(CASE ... THEN 1 END)` with no `ELSE`: unmatched rows become NULL and `COUNT` skips them. Using `ELSE 0` there would count everything.

3. Joins

```
SELECT h.SalesOrderID, d.ProductID, p.Name
FROM Sales.SalesOrderHeader h
INNER JOIN Sales.SalesOrderDetail d ON h.SalesOrderID = d.SalesOrderID
LEFT JOIN Production.Product p ON d.ProductID = p.ProductID
WHERE h.OrderDate >= '2013-01-01';
```

Join	Returns
INNER	Only matching rows from both
LEFT [OUTER]	All left rows; right columns NULL when unmatched
RIGHT [OUTER]	All right rows
FULL [OUTER]	All rows from both
CROSS	Cartesian product
SELF	A table joined to itself with aliases

The LEFT JOIN + WHERE trap

```
-- BROKEN: this silently becomes an INNER join
SELECT c.CustomerID, o.SalesOrderID
FROM Customer c LEFT JOIN SalesOrderHeader o ON c.CustomerID = o.CustomerID
WHERE o.OrderDate >= '2014-01-01';

-- CORRECT: put the right-table condition in the ON clause
FROM Customer c LEFT JOIN SalesOrderHeader o
ON c.CustomerID = o.CustomerID AND o.OrderDate >= '2014-01-01';
```

Unmatched rows have `o.OrderDate = NULL`, and `NULL >= '2014-01-01'` is unknown, so `WHERE` discards them. **Conditions on the right table of a LEFT JOIN belong in ON; conditions on the left table belong in WHERE.**

Finding non-matches

```
SELECT c.CustomerID FROM Customer c
LEFT JOIN SalesOrderHeader o ON c.CustomerID = o.CustomerID
WHERE o.CustomerID IS NULL; -- anti-join idiom

SELECT CustomerID FROM Customer c
WHERE NOT EXISTS (SELECT 1 FROM SalesOrderHeader o
WHERE o.CustomerID = c.CustomerID); -- usually preferred
```

Self-join

```
SELECT e.BusinessEntityID, e.JobTitle, m.JobTitle AS ManagerTitle
FROM HumanResources.Employee e
LEFT JOIN HumanResources.Employee m ON e.ManagerID = m.BusinessEntityID;
```

For pair-wise self-joins use `a.id < b.id` rather than `<>` — otherwise every pair appears twice.

Join fan-out: if the right table's key isn't unique, rows multiply and every `SUM` is inflated. Check `SELECT key, COUNT(*) FROM dim GROUP BY key HAVING COUNT(*) > 1` before joining, and verify the row count is unchanged after.

4. Aggregation

```

SELECT TerritoryID,
       COUNT(*)           AS Orders,
       COUNT(DISTINCT CustomerID) AS Customers,
       SUM(TotalDue)       AS Revenue,
       AVG(TotalDue)       AS AvgOrder,
       MIN(OrderDate)      AS FirstOrder,
       MAX(TotalDue)       AS LargestOrder
FROM Sales.SalesOrderHeader
WHERE OrderDate >= '2013-01-01' -- filters ROWS, before grouping
GROUP BY TerritoryID
HAVING SUM(TotalDue) > 1000000 -- filters GROUPS, after
ORDER BY Revenue DESC;

```

Rule: every non-aggregated column in `SELECT` must appear in `GROUP BY`. (MySQL is lenient by default; standard SQL and SQL Server are not.)

```

GROUP BY ROLLUP(Region, City) -- hierarchical subtotals + grand total
GROUP BY CUBE(Region, Channel) -- every combination
GROUP BY GROUPING SETS ((Region), (Channel), ())

```

5. Subqueries

Scalar — returns one value:

```

SELECT Name, ListPrice,
       (SELECT AVG(ListPrice) FROM Production.Product) AS AvgPrice
FROM Production.Product
WHERE ListPrice > (SELECT AVG(ListPrice) FROM Production.Product);

```

Multi-row with `IN` / `ANY` / `ALL` :

```

WHERE TerritoryID IN (SELECT TerritoryID FROM Sales.SalesTerritory WHERE [Group] = 'Europe')
WHERE ListPrice > ALL (SELECT ListPrice FROM Product WHERE Colour = 'Red')

```

Correlated — references the outer query, conceptually re-evaluated per row:

```

SELECT p.Name, p.ListPrice
FROM Production.Product p
WHERE p.ListPrice > (SELECT AVG(p2.ListPrice) FROM Production.Product p2
                   WHERE p2.ProductSubcategoryID = p.ProductSubcategoryID);

```

EXISTS — stops at the first match, so it's usually faster than `IN` and is NULL-safe:

```

WHERE EXISTS (SELECT 1 FROM SalesOrderDetail d WHERE d.ProductID = p.ProductID)
WHERE NOT EXISTS (...) -- prefer over NOT IN

```

Derived table (subquery in `FROM`) — must be aliased:

```

SELECT t.TerritoryID, t.Revenue
FROM (SELECT TerritoryID, SUM(TotalDue) AS Revenue
      FROM Sales.SalesOrderHeader GROUP BY TerritoryID) AS t
WHERE t.Revenue > 1000000;

```

6. CTEs

```

WITH TerritoryRevenue AS (
  SELECT TerritoryID, SUM(TotalDue) AS Revenue
  FROM Sales.SalesOrderHeader GROUP BY TerritoryID
),
Ranked AS (
  SELECT *, RANK() OVER (ORDER BY Revenue DESC) AS rnk
  FROM TerritoryRevenue
)
SELECT * FROM Ranked WHERE rnk <= 5;

```

- Readable top-to-bottom instead of inside-out.
- A later CTE can reference an earlier one — this is how you filter on a window function.
- Referenced multiple times in one query without repeating the text.

- A CTE is **not** automatically materialised; it's usually inlined by the optimiser. It is not a performance feature per se.

Recursive CTE — for hierarchies (org charts, BOM explosions):

```
WITH OrgChart AS (
    SELECT BusinessEntityID, ManagerID, 0 AS Depth      -- anchor
    FROM HumanResources.Employee WHERE ManagerID IS NULL
    UNION ALL
    SELECT e.BusinessEntityID, e.ManagerID, oc.Depth + 1 -- recursive part
    FROM HumanResources.Employee e
    JOIN OrgChart oc ON e.ManagerID = oc.BusinessEntityID
)
SELECT * FROM OrgChart OPTION (MAXRECURSION 100);
```

Structure: anchor member, `UNION ALL`, recursive member referencing the CTE. Always bound the recursion.

7. Window Functions

```
function() OVER (
    PARTITION BY col      -- optional: reset per group
    ORDER BY col          -- required for ranking and offsets
    ROWS BETWEEN ... AND ...-- optional frame
)
```

Unlike `GROUP BY`, windows **do not collapse rows** — every input row is returned with an extra computed column.

Ranking

Function	100, 90, 90, 80 →
<code>ROW_NUMBER()</code>	1, 2, 3, 4 (ties broken arbitrarily)
<code>RANK()</code>	1, 2, 2, 4 (ties share, then skip)
<code>DENSE_RANK()</code>	1, 2, 2, 3 (ties share, no skip)
<code>NTILE(4)</code>	Splits into 4 near-equal buckets
<code>PERCENT_RANK()</code>	$(\text{rank}-1)/(\text{n}-1)$

Offset

```
LAG(Revenue, 1) OVER (PARTITION BY Territory ORDER BY Yr)
LEAD(Revenue, 1, 0) OVER (...)      -- third arg = default when absent
FIRST_VALUE(x) OVER (...)
LAST_VALUE(x) OVER (... ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)
```

LAST_VALUE trap: the default frame is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, so `LAST_VALUE` returns the *current* row unless you specify `UNBOUNDED FOLLOWING`. Identical to the Spark trap — same spec, same bug.

Aggregate windows

```
SUM(TotalDue) OVER (PARTITION BY TerritoryID)      -- group total on every row
SUM(TotalDue) OVER (PARTITION BY T ORDER BY D
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) -- running total
AVG(TotalDue) OVER (ORDER BY D ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) -- 3-row moving avg
TotalDue * 100.0 / SUM(TotalDue) OVER (PARTITION BY TerritoryID)      -- share of group
```

ROWS vs RANGE : `ROWS` counts physical rows; `RANGE` groups by the ordering *value*, so tied values are all included in the same frame. Default is `RANGE`, which surprises people when there are ties.

Canonical patterns

```
-- Top N per group
WITH r AS (SELECT *, ROW_NUMBER() OVER (PARTITION BY Cat ORDER BY Rev DESC) rn FROM t)
SELECT * FROM r WHERE rn <= 3;

-- Period-over-period growth
SELECT Yr, Rev,
       Rev - LAG(Rev) OVER (ORDER BY Yr) AS Change,
       100.0*(Rev - LAG(Rev) OVER (ORDER BY Yr)) / LAG(Rev) OVER (ORDER BY Yr) AS PctChange
FROM Yearly;

-- Deduplicate keeping the latest
WITH d AS (SELECT *, ROW_NUMBER() OVER (PARTITION BY ID ORDER BY UpdatedAt DESC) rn FROM t)
SELECT * FROM d WHERE rn = 1;

-- Rows above their group average
WITH a AS (SELECT *, AVG(Price) OVER (PARTITION BY Cat) AS CatAvg FROM Product)
SELECT * FROM a WHERE Price > CatAvg;
```

That last one is the answer to “you can’t put a window function in `WHERE`” — the CTE layer is mandatory.

8. Set Operations

```
SELECT a FROM t1 UNION      SELECT a FROM t2; -- deduplicates (slower)
SELECT a FROM t1 UNION ALL  SELECT a FROM t2; -- keeps duplicates (faster) – default choice
SELECT a FROM t1 INTERSECT SELECT a FROM t2;
SELECT a FROM t1 EXCEPT   SELECT a FROM t2; -- MINUS in Oracle
```

Column count and compatible types must match. `ORDER BY` goes once, at the very end.

9. Common Functions

```
-- Strings
LEN(s) / LENGTH(s); LEFT(s,n); RIGHT(s,n); SUBSTRING(s, start, len)
UPPER(s); LOWER(s); TRIM(s); LTRIM(s); RTRIM(s)
REPLACE(s, old, new); CONCAT(a, b); a + b (SQL Server) / a || b (ANSI)
CHARINDEX(sub, s) / POSITION(sub IN s)

-- Dates (SQL Server flavour)
GETDATE(); DATEADD(day, 7, d); DATEDIFF(day, a, b)
YEAR(d); MONTH(d); DAY(d); DATEPART(quarter, d); DATENAME(month, d)
EOMONTH(d); CAST(d AS DATE); FORMAT(d, 'yyyy-MM')

-- Numeric
ROUND(x, 2); CEILING(x); FLOOR(x); ABS(x); POWER(x, 2); SQRT(x)

-- Conversion
CAST(x AS DECIMAL(10,2)); CONVERT(VARCHAR, d, 23); TRY_CAST(x AS INT)
```

10. Optimization

The word “optimize” is in your outcome statement. Know these.

Indexes. A clustered index defines the physical row order (one per table); non-clustered indexes are separate structures with pointers. A **covering index** includes all columns a query needs, avoiding a lookup back to the table.

Sargability — a predicate is “sargable” if an index can be used. Wrapping the column in a function kills index usage:

```
WHERE YEAR(OrderDate) = 2014 -- NOT sargable
WHERE OrderDate >= '2014-01-01' AND OrderDate < '2015-01-01' -- sargable

WHERE LEFT(Name, 3) = 'Bik' -- NOT sargable
WHERE Name LIKE 'Bik%' -- sargable ('%Bik%' is NOT – leading wildcard)
```

The rule: never wrap the indexed column in a function. Transform the constant instead.

Other principles: - `SELECT` only the columns you need — never `SELECT *` in production. - Filter as early and as selectively as possible. - `EXISTS` over `IN` for large subqueries; `NOT EXISTS` over `NOT IN` always (NULL safety). - `UNION ALL` over `UNION` unless you actually need dedup. - Beware implicit type conversion — comparing a `VARCHAR` column to an `INT` can force a scan. - Read the execution plan: look for table/index **scans** where a **seek** was expected, and for large discrepancies between estimated and actual row counts (usually stale statistics).

11. DDL / DML Basics

```
CREATE TABLE Sales.Summary (
    ID INT IDENTITY(1,1) PRIMARY KEY,
    Region VARCHAR(50) NOT NULL,
    Revenue DECIMAL(18,2) DEFAULT 0,
    CreatedAt DATETIME DEFAULT GETDATE(),
    CONSTRAINT FK_T FOREIGN KEY (TerritoryID) REFERENCES Sales.SalesTerritory(TerritoryID),
    CONSTRAINT CK_Rev CHECK (Revenue >= 0)
);

INSERT INTO t (a, b) VALUES (1, 'x'), (2, 'y');
INSERT INTO t (a, b) SELECT a, b FROM other;
UPDATE t SET b = 'z' WHERE a = 1;
DELETE FROM t WHERE a = 1;
TRUNCATE TABLE t;           -- fast, no WHERE, minimal logging, resets IDENTITY

MERGE INTO target AS t
USING source AS s ON t.id = s.id
WHEN MATCHED THEN UPDATE SET t.v = s.v
WHEN NOT MATCHED THEN INSERT (id, v) VALUES (s.id, s.v);
```

DELETE vs **TRUNCATE** vs **DROP** : delete rows (logged, can filter, keeps structure) / remove all rows (fast, no filter, keeps structure) / remove the table entirely.

12. Practice Checklist — AdventureWorks

- ☐ Total revenue by territory and year, only territories above a threshold
- ☐ Top 5 products by revenue **within each** category (**ROW_NUMBER** + CTE)
- ☐ Customers who have never placed an order (**NOT EXISTS**)
- ☐ Employees with their manager's name (self-join)
- ☐ YoY growth % per territory (**LAG**)
- ☐ Running total of sales by date (framed window)
- ☐ Each product's price vs its subcategory average (correlated subquery, then again as a window)
- ☐ Pivot years into columns using conditional aggregation
- ☐ Recursive CTE over the employee hierarchy with depth
- ☐ Take a query using **YEAR(col) = 2014** and rewrite it sargably
- ☐ Explain the difference between **WHERE** and **HAVING** with an example of each

ML, Time Series & Forecasting — Complete Reference

EY Probation Training — Week 2 (ML intro, time series, feature engineering, forecasting models, forecast evaluation) and Week 3 (FEU forecasting KPIs). Dataset: Kaggle Superstore.

The whole roadmap is **forecasting-shaped**. This is not general ML — it is regression-based demand forecasting with supply-chain KPIs. Frame everything through that lens.

1. Supervised Learning Basics

Supervised = labelled data. Regression predicts a continuous value; classification predicts a category. Forecasting is regression with a time dimension and ordering constraints.

Unsupervised = no labels (clustering, PCA, anomaly detection).

Bias-variance tradeoff

- **High bias / underfitting** — model too simple, poor on both train and test. Fix: more features, more capacity, less regularisation.
- **High variance / overfitting** — model memorises noise, great on train and poor on test. Fix: more data, fewer features, more regularisation, simpler model, early stopping.

Diagnose by comparing train and validation error. Small gap and both bad = bias. Large gap = variance.

The core workflow

1. Frame the problem and choose the metric **before** modelling.
2. Split the data (chronologically, for time series).
3. Explore and clean.
4. Engineer features.
5. Fit a **baseline first**.
6. Train candidate models.
7. Evaluate on held-out data with the chosen metric.
8. Interpret, then iterate.

Never skip step 5. A model that can't beat naive-forecast or seasonal-naive isn't worth deploying, and the baseline comparison is the first thing a reviewer asks for.

2. Regression Models

Linear regression

Fits $y = b_0 + b_1x_1 + \dots + b_nx_n$ by minimising squared error. Fast, interpretable coefficients, but assumes linearity and additivity, and is sensitive to outliers and multicollinearity.

Regularised variants: - **Ridge (L2)** — shrinks coefficients toward zero, handles multicollinearity, never zeroes them out. - **Lasso (L1)** — can drive coefficients to exactly zero, so it performs feature selection. - **ElasticNet** — a mix of both.

Tree-based models

Decision tree — recursive splits minimising impurity (MSE for regression). Interpretable but high variance; prone to overfitting unless depth-limited.

Random Forest — many trees on **bootstrap samples**, each split considering a **random subset of features**, predictions averaged. Bagging: reduces variance through averaging. Trees are built independently and in parallel.

Gradient Boosting / XGBoost — trees built **sequentially**, each fitting the residual errors of the ensemble so far. Reduces bias. More accurate than RF on tabular data but more sensitive to hyperparameters and easier to overfit.

	Random Forest	XGBoost
Trees built	Independently, in parallel	Sequentially, on residuals
Reduces	Variance (bagging)	Bias (boosting)
Overfitting risk	Low	Higher — needs tuning and early stopping
Tuning sensitivity	Forgiving	Sensitive
Handles NaN natively	No (sklearn)	Yes
Typical use	Strong quick baseline	Squeezing out accuracy

Key hyperparameters:

```
RandomForestRegressor(n_estimators=300, max_depth=None, min_samples_leaf=5,
                      max_features="sqrt", n_jobs=-1, random_state=42)

XGBRegressor(n_estimators=1000, learning_rate=0.05, max_depth=6,
             subsample=0.8, colsample_bytree=0.8,
             reg_alpha=0, reg_lambda=1, early_stopping_rounds=50,
             random_state=42)
```

Lower `learning_rate` needs more `n_estimators`. `subsample` and `colsample_bytree` add randomness that fights overfitting. Use `early_stopping_rounds` with a validation set rather than guessing `n_estimators`.

Feature importance: tree-based `.feature_importances_` is impurity-based and biased toward high-cardinality features. **Permutation importance** and **SHAP** are more trustworthy. Importance is not causation.

3. Time Series Fundamentals

Components

- **Trend** — long-run direction.
- **Seasonality** — a repeating pattern of **fixed, known period** (weekly, monthly, yearly).
- **Cyclic** — repeating but of **variable, unknown length** (business cycles). Not the same as seasonality; a common exam distinction.
- **Residual / irregular** — what's left.

Decomposition: - **Additive:** $y = \text{Trend} + \text{Seasonality} + \text{Residual}$ — when seasonal swings are roughly constant in absolute size. -

Multiplicative: $y = \text{Trend} \times \text{Seasonality} \times \text{Residual}$ — when swings grow with the level. A log transform converts multiplicative to additive.

```
from statsmodels.tsa.seasonal import seasonal_decompose, STL
seasonal_decompose(series, model="additive", period=12).plot()
STL(series, period=12).fit().plot()      # more robust to outliers
```

Stationarity

A series is stationary if mean, variance and autocovariance don't change over time. ARIMA-family models require it.

Testing: - **ADF (Augmented Dickey-Fuller):** H_0 = has a unit root = **non-stationary**. $p < 0.05 \rightarrow$ reject $H_0 \rightarrow$ stationary. - **KPSS:** H_0 = **stationary** — the opposite null. $p < 0.05 \rightarrow$ non-stationary.

Getting the null hypotheses the right way round is a classic exam trap. Running both and agreeing is best practice.

Making a series stationary: differencing ($y_t - y_{t-1}$), seasonal differencing ($y_t - y_{t-s}$), log or Box-Cox transform for non-constant variance, or explicit de-trending.

ACF and PACF

- **ACF** — correlation of the series with its own lags, including indirect effects. Slow decay indicates a trend (non-stationary). Spikes at multiples of s indicate seasonality.
- **PACF** — correlation at lag k with the effects of intermediate lags removed.

Pattern	Suggests
ACF cuts off after lag q , PACF tails off	MA(q)
PACF cuts off after lag p , ACF tails off	AR(p)
Both tail off	ARMA(p, q)
ACF decays very slowly	Non-stationary — difference it

4. Univariate Models

ARIMA(p, d, q) — p autoregressive terms, d differences, q moving-average terms. **SARIMA(p, d, q)(P, D, Q) s** — adds a seasonal component with period s .

```
from statsmodels.tsa.statespace.sarimax import SARIMAX
m = SARIMAX(y, order=(1,1,1), seasonal_order=(1,1,1,12)).fit()
fc = m.get_forecast(steps=12); fc.predicted_mean; fc.conf_int()
```

Exponential smoothing (Holt-Winters) — weighted averages with exponentially decaying weights. - Simple ES: level only, no trend or seasonality. - Holt's linear: level + trend. - Holt-Winters: level + trend + seasonality (additive or multiplicative).

```
from statsmodels.tsa.holtwinters import ExponentialSmoothing
ExponentialSmoothing(y, trend="add", seasonal="add", seasonal_periods=12).fit()
```

Baselines you must always compute: - **Naive:** forecast = last observed value. - **Seasonal naive:** forecast = value from the same period last season. Usually the benchmark to beat in demand forecasting. - **Moving average / drift.**

5. Feature Engineering for Forecasting

This is the heart of Week 2 — ML-based forecasting turns a time series into a supervised tabular problem.

Lag features

```
for L in [1, 2, 3, 7, 14, 28]:
    df[f"lag_{L}"] = df.groupby("sku")["sales"].shift(L)
```

Choose lags by horizon. If you must forecast 7 days ahead, `lag 1` will not exist at prediction time — you'd be using data you won't have. Lags must be at least as large as the forecast horizon, or you need a recursive strategy.

Rolling / moving-average features

```
g = df.groupby("sku")["sales"]
df["roll_mean_7"] = g.transform(lambda s: s.shift(1).rolling(7).mean())
df["roll_std_7"] = g.transform(lambda s: s.shift(1).rolling(7).std())
df["roll_max_28"] = g.transform(lambda s: s.shift(1).rolling(28).max())
df["ewm_7"] = g.transform(lambda s: s.shift(1).ewm(span=7).mean())
```

The `.shift(1)` before `.rolling()` is mandatory. Without it the window includes the current row — the value you're trying to predict — and that is leakage. Your validation score will look excellent and the model will fail in production. This is the single most common forecasting bug.

Date-based features

```
d = df["date"].dt
df["year"], df["month"], df["day"] = d.year, d.month, d.day
df["dayofweek"], df["weekofyear"], df["quarter"] = d.dayofweek, d.isocalendar().week, d.quarter
df["is_weekend"] = d.dayofweek >= 5
df["is_month_start"] = d.is_month_start
df["days_since_start"] = (df.date - df.date.min()).dt.days # linear trend proxy
# cyclical encoding - so December is adjacent to January
df["month_sin"] = np.sin(2*np.pi*d.month/12)
df["month_cos"] = np.cos(2*np.pi*d.month/12)
```

Cyclical encoding matters: as a plain integer, month 12 and month 1 look eleven apart when they're actually adjacent.

Other useful features

Holiday and promotion flags; price and discount; days since last promotion; expanding mean; group-level aggregates (category mean, store mean); and **target encoding of categoricals — computed on the training fold only.**

6. Train/Test Splitting for Time Series

Never use a random split or plain K-fold. A random split lets the model see t+1 while predicting t. Split **chronologically**.

```
cutoff = df.date.max() - pd.Timedelta(days=90)
train, test = df[df.date <= cutoff], df[df.date > cutoff]
```

Walk-forward / rolling-origin validation:

```
from sklearn.model_selection import TimeSeriesSplit
for tr_idx, te_idx in TimeSeriesSplit(n_splits=5).split(X):
    ...
```

Each fold trains on everything up to a point and tests on the next block. Expanding window keeps all history; sliding window keeps a fixed recent span.

Leakage checklist

- Rolling/lag features must use `.shift()` so they exclude the current value.
- Scalers, imputers and encoders are **fit on train only**, then applied to test.
- No feature may use information unavailable at prediction time (e.g. end-of-month totals when predicting mid-month).
- With group structure (per SKU, per store), compute lags **within group** — a `shift` without `groupby` bleeds one SKU's history into another's.
- The test period must sit strictly after the train period, with a gap if the horizon requires it.

7. Forecast Evaluation

General regression metrics

Metric	Formula	Character
MAE	$\text{mean}(y - \hat{y})$	Same units as y; robust to outliers
MSE	$\text{mean}((y - \hat{y})^2)$	Penalises large errors heavily; squared units
RMSE	$\sqrt{\text{MSE}}$	Same units as y; outlier-sensitive
MAPE	$\text{mean}(y - \hat{y} / y) \times 100$	Scale-free %; undefined at y = 0 , asymmetric
sMAPE	symmetric variant	Handles zeros better
R ²	$1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$	Variance explained; can be negative

MAE vs RMSE: RMSE > MAE always (equal only when all errors are identical). A large gap between them indicates a few big errors. Choose MAE when all errors cost proportionally; RMSE when large misses are disproportionately costly (usually true in inventory).

MAPE's problems are worth knowing: it explodes near zero demand, and it penalises over-forecasting more than under-forecasting, which biases model selection toward under-forecasting. In intermittent demand it's often unusable.

Supply-chain forecasting KPIs

These appear in both Week 2 and Week 3 — a strong signal they'll be assessed.

Forecast Accuracy (FA) — commonly $\text{FA} = (1 - \text{MAPE}) \times 100\%$, or the WMAPE-based $\text{FA} = (1 - \Sigma|A-F| / \Sigma A) \times 100\%$. The weighted form is preferred because it doesn't blow up on low-volume items and it weights by actual volume.

Forecast Bias (FB) — **signed**, unlike the error metrics:

$$\text{FB} = \Sigma(F - A) / \Sigma A \times 100\%$$

- Positive → systematically **over**-forecasting → excess inventory, obsolescence.
- Negative → systematically **under**-forecasting → stockouts, lost sales.
- Near zero → unbiased, though it can still have large offsetting errors.

Bias is the one that matters most operationally. A forecast can have excellent accuracy and terrible bias, or acceptable accuracy with zero bias. Bias compounds over time in inventory; random error largely cancels. Always report accuracy and bias **together** — reporting only accuracy is the mistake.

Tracking Signal = cumulative bias / MAD. Sustained values outside roughly ± 4 indicate the model needs re-fitting.

WMAPE / weighted MAPE = $\Sigma|A-F| / \Sigma A$. The standard workhorse for a portfolio of SKUs.

Get FEU's own formulas from their documentation. Different organisations define FA and FB with different denominators (actuals vs forecast), sign conventions, and aggregation levels. Confidently giving the textbook version when theirs differs scores worse than asking.

Aggregation matters

Accuracy computed at SKU-store-day level is much worse than the same forecast measured at category-month level, because errors cancel on aggregation. Always state the level. Comparing two forecasts measured at different levels is meaningless.

8. Worked Pipeline (Superstore)

```

import pandas as pd, numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error
from xgboost import XGBRegressor

df = pd.read_csv("superstore.csv", parse_dates=["Order Date"], encoding="latin-1")

# 1. aggregate to a regular grid
ts = (df.groupby([pd.Grouper(key="Order Date", freq="W"), "Category"])["Sales"]
      .sum().reset_index().rename(columns={"Order Date": "date", "Sales": "sales"}))

# 2. complete the spine so gaps are explicit
spine = (pd.MultiIndex.from_product(
    [pd.date_range(ts.date.min(), ts.date.max(), freq="W"), ts.Category.unique()],
    names=["date", "Category"]).to_frame(index=False))
ts = spine.merge(ts, on=["date", "Category"], how="left").fillna({"sales": 0})
ts = ts.sort_values(["Category", "date"])

# 3. features – note shift(1) before every rolling window
g = ts.groupby("Category")["sales"]
for L in [1, 2, 4, 8, 52]:
    ts[f"lag_{L}"] = g.shift(L)
for W in [4, 8]:
    ts[f"roll_mean_{W}"] = g.transform(lambda s: s.shift(1).rolling(W).mean())
    ts[f"roll_std_{W}"] = g.transform(lambda s: s.shift(1).rolling(W).std())
ts["month"] = ts.date.dt.month
ts["week"] = ts.date.dt.isocalendar().week.astype(int)
ts["msin"], ts["mcos"] = np.sin(2*np.pi*ts.month/12), np.cos(2*np.pi*ts.month/12)
ts = ts.dropna()

# 4. chronological split
cut = ts.date.max() - pd.Timedelta(weeks=13)
train, test = ts[ts.date <= cut], ts[ts.date > cut]
feats = [c for c in ts.columns if c not in ("date", "Category", "sales")]
Xtr, ytr, Xte, yte = train[feats], train.sales, test[feats], test.sales

# 5. baselines FIRST
naive = test.groupby("Category")["lag_1"]
print("naive MAE:", mean_absolute_error(yte, test["lag_1"]))
print("seasonal naive MAE:", mean_absolute_error(yte, test["lag_52"]))

# 6. models
rf = RandomForestRegressor(n_estimators=300, min_samples_leaf=3,
                           random_state=42, n_jobs=-1).fit(Xtr, ytr)
xgb = XGBRegressor(n_estimators=600, learning_rate=0.05, max_depth=5,
                   subsample=0.8, colsample_bytree=0.8, random_state=42).fit(Xtr, ytr)

# 7. evaluate – accuracy AND bias together
def report(name, y, p):
    mae = mean_absolute_error(y, p)
    rmse = np.sqrt(mean_squared_error(y, p))
    wmape = np.abs(y - p).sum() / y.sum() * 100
    bias = (p - y).sum() / y.sum() * 100
    print(f"{name:8s} MAE {mae:9.1f} RMSE {rmse:9.1f} "
          f"WMAPE {wmape:6.2f}% FA {100-wmape:6.2f}% Bias {bias:+6.2f}%")

report("RF", yte, rf.predict(Xte))
report("XGB", yte, xgb.predict(Xte))

```

Report FA and bias side by side, and always against the baselines.

9. Practice Checklist

- ☐ Decompose a series; state additive vs multiplicative and why
- ☐ Run ADF and KPSS; state both null hypotheses correctly
- ☐ Read ACF/PACF plots and propose ARIMA orders
- ☐ Build lag, rolling and date features **with** the `.shift(1)` guard
- ☐ Explain three concrete leakage risks in a forecasting pipeline
- ☐ Chronological split plus `TimeSeriesSplit` walk-forward
- ☐ Fit naive and seasonal-naive baselines before any model
- ☐ Fit RF and XGBoost; explain bagging vs boosting
- ☐ Compute MAE, RMSE, MAPE, WMAPE, FA and FB on the same forecast

- ☐ Explain why bias matters more than accuracy operationally
- ☐ Show how the same forecast scores differently at two aggregation levels

Azure, ADF, Databricks & Code Industrialization — Reference

EY Probation Training — Week 4 (FEU Technical Architecture). Covers Azure infrastructure, ADF & Databricks, Azure DevOps & code industrialization, backtesting and parallel run.

Week 4 mixes **generic platform knowledge** (in this document) with **FEU-specific configuration** (only obtainable from their KT sessions and SharePoint). Learn the generic layer here so that in the sessions you're mapping their choices onto concepts you already hold, rather than meeting both at once.

1. Azure Data Platform Landscape

Service	Role
ADLS Gen2	Hierarchical data lake storage; the usual landing and curated zones
Azure Data Factory (ADF)	Orchestration and ELT — pipelines, triggers, copy activities
Azure Databricks	Spark compute; notebooks, jobs, Delta Lake
Azure Synapse	Analytics warehouse; SQL pools
Azure Key Vault	Secrets, keys, certificates
Azure DevOps	Repos, Pipelines, Boards, Artifacts
Azure Monitor / Log Analytics	Metrics, logs, alerting
Microsoft Entra ID	Identity (formerly Azure AD)

Medallion architecture

The near-universal lakehouse layout, and almost certainly how FEU is organised:

- **Bronze** — raw ingested data, append-only, schema-on-read, full history. Never edited.
- **Silver** — cleaned, deduplicated, conformed, joined. Business entities.
- **Gold** — aggregated, business-level tables and feature sets ready for consumption.

The value is reprocessability: if Silver logic is wrong, you rebuild it from Bronze without re-ingesting from source.

Identity and access

- **Service Principal** — an identity for an application, used for non-interactive pipeline auth.
- **Managed Identity** — an Azure-managed service principal with no credentials to store. Preferred where supported.
- **RBAC** — role assignments at subscription/resource-group/resource scope.
- **Key Vault** — never hard-code secrets. ADF and Databricks both read from Key Vault; in Databricks it appears as a secret scope.

2. Azure Data Factory

Core concepts

Concept	Meaning
Pipeline	A container of activities, forming a workflow
Activity	One step — Copy, Notebook, Stored Procedure, Lookup, ForEach, If
Dataset	A named view of the data a activity reads or writes
Linked Service	The connection string / credential to a data store or compute
Integration Runtime (IR)	The compute that runs the activity
Trigger	What starts a pipeline

Integration Runtime types

- **Azure IR** — fully managed, for cloud-to-cloud movement and data flows.
- **Self-hosted IR** — installed on a VM or on-premises machine, for accessing private networks and on-prem sources.
- **Azure-SSIS IR** — lift-and-shift for existing SSIS packages.

Trigger types

- **Schedule** — wall-clock recurrence.
- **Tumbling window** — fixed-size, non-overlapping, contiguous time slices, with dependency and backfill support. This is what you want for reliable incremental loads.
- **Event-based** — fires on blob created/deleted.
- **Manual / on-demand**.

Schedule vs tumbling window is a likely question: tumbling windows retain state, can be re-run for a historical slice, and support dependencies between windows; schedule triggers are fire-and-forget.

Control flow activities

`ForEach` (with an optional `batchCount` for parallelism), `If Condition`, `Switch`, `Until`, `Lookup` (retrieve a config row or list), `Get Metadata`, `Execute Pipeline`, `Web`, `Wait`, `Fail`, `Set/Append Variable`.

Parameterisation — the difference between a maintainable factory and 200 near-identical pipelines. Parameterise datasets and linked services, drive a `ForEach` from a metadata/config table, and pass parameters through `@pipeline().parameters.x`. Expressions use the ADF expression language: `@concat()`, `@utcnow()`, `@formatDateTime()`, `@activity('X').output`.

Mapping Data Flows

Visual, code-free transformations that compile to Spark and run on the Azure IR. Useful for teams without Spark skills; most engineering teams prefer Databricks notebooks called from ADF, because notebooks are versionable, testable and reviewable as code.

Monitoring and reliability

- Monitor tab shows pipeline and activity runs; re-run from a failed activity.
- Per-activity **retry** count and interval; **timeout**.
- Alerts via Azure Monitor.
- **Idempotency matters**: a re-run must not double-load. Use overwrite semantics, a watermark, or a Delta `MERGE`.

3. Databricks

Workspace concepts

- **Cluster** — the Spark compute. **All-purpose** clusters are interactive and shared; **job** clusters are created for a single run and terminated after, which is cheaper and better isolated.
- **Notebook** — cells in Python/SQL/Scala/R. `%python`, `%sql`, `%md`, `%sh`, `%run ./other_notebook`, `%pip install`.
- **Job / Workflow** — scheduled or triggered execution of notebooks/JARs/wheels with task dependencies.
- **DBFS** — the file system abstraction. Prefer mounting ADLS or, better, Unity Catalog external locations.
- **Unity Catalog** — centralised governance: three-level namespace `catalog.schema.table`, lineage, access control.
- **Repos** — Git integration so notebooks live in version control.

```
dbutils.fs.ls("/mnt/data")
dbutils.widgets.text("run_date", "")           # parameters from ADF/Jobs
dbutils.notebook.exit("success")               # return a value to the caller
dbutils.secrets.get(scope="kv", key="sql-pw")  # never print this
```

Cluster sizing basics

Driver holds the plan and collected results; workers do the work. Autoscaling adjusts worker count within bounds. **Photon** is a vectorised C++ engine that accelerates SQL and DataFrame workloads. Spot/low-priority instances cut cost but can be reclaimed — fine for fault-tolerant batch, not for a run with a tight SLA.

Delta Lake

```
MERGE INTO gold.sales t USING staging.sales s ON t.id = s.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;

DESCRIBE HISTORY gold.sales;
SELECT * FROM gold.sales VERSION AS OF 12;      -- time travel
OPTIMIZE gold.sales ZORDER BY (customer_id);    -- compaction + data skipping
VACUUM gold.sales RETAIN 168 HOURS;
```

Delta gives ACID transactions, schema enforcement and evolution, time travel, and upsert. `OPTIMIZE` fixes the small-files problem; `ZORDER` co-locates related values so more files can be skipped.

ADF + Databricks together

The standard pattern: ADF orchestrates (scheduling, dependencies, retries, alerting) and calls Databricks notebooks via the **Databricks Notebook activity**, passing parameters as widgets. Databricks does the compute. ADF handles the control plane; Databricks handles the data plane.

4. Azure DevOps & Code Industrialization

“Code industrialization” means moving from notebooks-on-someone’s-laptop to versioned, reviewed, tested, automatically deployed code.

Repos and branching

Git flow variants you’ll meet: - `main` / `master` — production, protected. - `develop` — integration branch. - `feature/<ticket>-description` — one branch per unit of work. - `release/x.y`, `hotfix/...`

Branch policies on protected branches: require a pull request, minimum number of reviewers, linked work item, build must pass, no direct pushes.

Pull request etiquette: small and focused; a description explaining *why*, not just *what*; self-review before requesting others; respond to every comment; never merge your own PR where policy forbids it.

Merge strategies: merge commit (preserves history), squash (one tidy commit per PR — common), rebase (linear history, rewrites

commits).

```
git checkout -b feature/FEU-123-add-lag-features
git add -p && git commit -m "feat: add lag and rolling features to forecast prep"
git push -u origin feature/FEU-123-add-lag-features
git fetch origin && git rebase origin/develop
```

Commit message convention (Conventional Commits): `feat:`, `fix:`, `refactor:`, `docs:`, `test:`, `chore:`. Imperative mood, under ~72 characters in the subject.

Repository structure

A defensible layout for a data/ML project:

```
project/
  src/           package code – importable, testable
    ingestion/
    transforms/
    features/
    models/
  notebooks/     exploration; thin, calling into src/
  tests/         unit and integration tests mirroring src/
  config/        environment configs (no secrets)
  pipelines/     ADF/Databricks job definitions as JSON/YAML
  docs/
  requirements.txt / pyproject.toml
  README.md
  .gitignore
```

The principle: notebooks call functions; they don't contain the logic. Logic in `src/` can be unit-tested, reviewed as a diff, and reused. Logic buried in a notebook cell cannot.

CI/CD pipelines

```
trigger:
  branches: { include: [develop, main] }

pool: { vmImage: 'ubuntu-latest' }

steps:
  - task: UsePythonVersion@0
    inputs: { versionSpec: '3.11' }
  - script: pip install -r requirements.txt
    displayName: Install
  - script: ruff check src/ && black --check src/
    displayName: Lint
  - script: pytest tests/ --cov=src --cov-report=xml
    displayName: Test
  - task: PublishTestResults@2
```

CI = on every push/PR, build, lint and test automatically. **CD** = deploy the validated artefact to Dev → Test/UAT → Prod, with approval gates between environments. Infrastructure and pipeline definitions are stored as code (ARM/Bicep/Terraform, ADF JSON) so environments are reproducible.

Environment promotion: the same artefact moves through environments; only configuration changes. Never rebuild per environment — that defeats the purpose of testing the artefact.

Coding standards

- **PEP 8** and a formatter (`black`) plus a linter (`ruff` / `flake8`) enforced in CI, so style is never a review topic.
- **Naming:** `snake_case` functions and variables, `PascalCase` classes, `UPPER_SNAKE` constants. Descriptive over short.
- **Docstrings** on every public function — purpose, args, returns, raises.
- **Type hints** on function signatures.
- **No hard-coded paths, credentials or magic numbers** — use config and Key Vault.
- **Logging, not print** . Use the `logging` module with levels; logs need timestamps and context to be useful in a job run.
- **Error handling:** catch specific exceptions, fail loudly on data-quality violations, and make failures diagnosable.
- **DRY**, small functions, single responsibility.
- **Unit tests** for transformation logic; `chispa` or comparing collected rows for Spark DataFrames.

```

import logging
logger = logging.getLogger(__name__)

def add_lag_features(df: DataFrame, cols: list[str], lags: list[int]) -> DataFrame:
    """Add lagged versions of the given columns, partitioned by SKU.

    Args:
        df: input frame, must contain 'sku' and 'date'.
        cols: columns to lag.
        lags: lag offsets in periods.
    Returns:
        DataFrame with one new column per (col, lag) pair.
    Raises:
        ValueError: if a required column is missing.
    """
    missing = {"sku", "date"} - set(df.columns)
    if missing:
        raise ValueError(f"missing required columns: {missing}")
    logger.info("adding %d lag features", len(cols) * len(lags))
    ...

```

Data quality gates

Assert on the things that silently corrupt results: row-count preservation across a 1:1 join, primary-key uniqueness, null rates within expected bounds, referential integrity, and value ranges. Fail the job rather than writing bad data downstream. Tools: Great Expectations, [dbt](#) tests, Delta constraints, or plain asserts in the job.

5. Backtesting and Parallel Run

Week 4's operational topic. Both are about proving a change is safe before it takes over.

Backtesting

Running the forecasting model over **historical** periods as if it were live, to estimate the accuracy you'd have achieved. Key requirements: - Use only information available at each historical decision point — **no lookahead**. - Use rolling-origin evaluation, not a single split. - Report the same KPIs the business uses (FA, bias) at the same aggregation level. - Compare against the incumbent model and against naive baselines.

STP in this context most likely means straight-through processing — the proportion of forecasts that flow through without manual intervention. Confirm the definition with the team; acronyms vary by organisation.

Parallel run

Running the new system **alongside** the existing one on live data, both producing output, with only the incumbent's output used for decisions. You compare them over an agreed window before cutting over.

- Purpose: catch discrepancies under real conditions, including data issues that never appear in historical backtests.
- Exit criteria should be agreed **before** starting: what tolerance of difference is acceptable, over what period, on which metrics.
- Investigate every material discrepancy — differences are information, not noise to be averaged away.

This is the same idea as a **shadow deployment** in software: new code receives production traffic, its output is logged and compared, but it doesn't serve users.

Run monitoring and support

Job success/failure alerting, SLA tracking (did the forecast land before the planning cycle needs it), data freshness checks, drift monitoring on inputs and on accuracy, runbooks for common failures, and an on-call/escalation path. In a support rotation the questions are always: is it failing, is it late, is it wrong — and each needs different instrumentation.

6. Questions Worth Being Able to Answer

- Difference between all-purpose and job clusters, and when each is appropriate
- Schedule trigger vs tumbling-window trigger
- Why medallion architecture, and what specifically goes in Silver
- What Delta gives you over plain Parquet
- Why notebooks should call [src/](#) code rather than contain logic
- What CI actually runs, and what blocks a merge
- How secrets reach a Databricks notebook without appearing in code
- What makes a pipeline idempotent, and why re-runs demand it
- Backtesting vs parallel run — what each catches that the other doesn't
- How you'd detect that a forecast job silently produced wrong numbers

7. What to Capture in FEU KT Sessions

Generic knowledge won't cover these. Take notes specifically on:

- ☐ Which Azure subscriptions/resource groups, and how environments are separated

- ☐ Storage layout and zone naming — does it follow medallion, and what are the actual container names
- ☐ Which orchestration tool actually triggers production runs, and on what schedule
- ☐ Cluster configurations and who can create them
- ☐ Repo structure, branch naming, and the exact PR policy
- ☐ The CI pipeline's stages and what causes a build to fail
- ☐ Deployment path from dev to prod, and who approves
- ☐ The backtesting harness — where it lives, how it's invoked, what it reports
- ☐ Parallel run exit criteria used historically
- ☐ Where run failures are alerted, and the escalation path

FEU Domain KT — Concepts & Note-Taking Scaffold

EY Probation Training — Week 3 (FEU Domain KT) and parts of Week 4.

Read this differently from the other six documents. Everything else here is knowledge you can acquire independently. Week 3 is **internal domain knowledge** that exists only in FEU's SharePoint, their KT sessions, and their people's heads. No external source can supply it.

What this document does: gives you the **generic supply-chain forecasting vocabulary** so the sessions land on prepared ground, and provides a **structured set of questions** to fill in as you go. Treat the blanks as the actual deliverable.

1. Generic Demand-Forecasting Vocabulary

These concepts are industry-standard. FEU's implementation will differ in detail, but the underlying ideas are stable.

Planning hierarchy

Demand forecasting almost always operates over two intersecting hierarchies:

Product hierarchy — e.g. Item / SKU → Brand → Sub-category → Category → Total. **Location or customer hierarchy** — e.g. Store → Customer → Channel → Region → Total.

Forecasts are generated at one level and **aggregated up** or **disaggregated down**.

- **Top-down** — forecast the aggregate, split it using historical proportions. Stable but loses item-level signal.
- **Bottom-up** — forecast each item, sum upward. Captures detail but noisy at the item level.
- **Middle-out** — forecast at an intermediate level, then go both directions.
- **Reconciliation** — making the levels mathematically consistent after independent forecasts.

The general principle: aggregation reduces noise, so accuracy measured at a higher level always looks better. This is why the **level of measurement must always be stated**.

GTIN

Global Trade Item Number — the GS1 standard product identifier, the number behind a barcode. Variants: GTIN-8, GTIN-12 (UPC), GTIN-13 (EAN), GTIN-14 (case/carton level).

Important for forecasting because the same physical product can carry different GTINs across pack sizes and markets, and GTINs get retired and reissued. Any join on GTIN needs care about which level and which time period.

APG

Not a universal standard acronym — in planning contexts it most often means an **Aggregated Planning Group** or **Article Planning Group**: a grouping of products planned together, typically because they share demand behaviour, substitute for each other, or are too sparse to forecast individually.

Confirm FEU's exact definition. This is precisely the kind of term where assuming the generic meaning will make you sound confidently wrong.

Customer hierarchy

The buying-side structure — ship-to location → sold-to customer → customer group → channel → region. Retailers often have deep hierarchies where forecasts are agreed at a negotiated level.

Baseline vs total forecast

- **Baseline** — the underlying demand pattern excluding promotions, events and one-offs. Usually what the statistical model produces.
- **Uplift** — the incremental demand attributable to a promotion or event.
- **Total / final forecast** — baseline + uplift + manual overrides.

The distinction matters because baseline accuracy and final accuracy are different measurements, and a team can be good at one and poor at the other.

Forecast lifecycle

A typical cycle, which FEU's will resemble: 1. **Data ingestion** — sales history, master data, promotions calendar, price. 2. **Cleansing** — outlier treatment, removing the effect of stockouts (censored demand), history alignment for product transitions. 3. **Baseline generation** — statistical or ML model. 4. **Enrichment** — promotional uplift, new-product forecasting, judgemental overrides. 5. **Review / consensus** — demand planners adjust; sometimes a formal S&OP meeting. 6. **Publication** — the forecast is locked and passed to supply planning. 7. **Measurement** — accuracy and bias computed once actuals arrive, at agreed lags.

Lag and horizon

- **Horizon** — how far ahead you forecast (e.g. 13 weeks).
- **Lag** — how far in advance the forecast being measured was made. "Lag-1 accuracy" measures the forecast made one period before; "lag-3" measures the one made three periods before.

Accuracy always degrades with lag. **Comparing accuracy figures at different lags is meaningless**, and quoting a lag-1 number as if it were a lag-3 number is a classic way to overstate performance.

Other terms you may meet

- **Intermittent / lumpy demand** — many zero periods; standard metrics like MAPE break down. Croston's method and its variants exist for this.
- **Cannibalisation** — a new or promoted product taking demand from another.
- **Halo effect** — a promotion lifting demand for related products.
- **Phase-in / phase-out** — new product introduction and discontinuation; history must be linked across the transition.
- **Censored demand** — sales don't equal demand when you were out of stock. Uncorrected, this teaches the model that demand was lower than it was.
- **S&OP / IBP** — Sales & Operations Planning / Integrated Business Planning; the governance process forecasts feed into.

2. Forecasting KPIs — Recap

Covered fully in the ML & Forecasting document; repeated here because Week 3 revisits them from the business side.

- **Forecast Accuracy (FA)** — typically $(1 - \text{WMAPE}) \times 100\%$.
- **Forecast Bias (FB)** — **signed**; positive is over-forecasting, negative is under-forecasting.
- **WMAPE** — $\sum |A-F| / \sum A$; the volume-weighted workhorse.
- **Tracking signal** — cumulative bias over MAD; flags systematic drift.

Report **accuracy and bias together, always**. Accuracy alone hides systematic over- or under-forecasting, and bias is what actually accumulates into excess stock or stockouts.

The three questions to ask about any KPI figure quoted at FEU: 1. At what **hierarchy level** is it measured? 2. At what **lag**? 3. Is it **baseline** or **final** forecast?

Without those three, an accuracy number means nothing — and asking them makes you look like you understand the domain.

3. Note-Taking Scaffold

Fill these in during KT sessions. Blank entries are gaps to chase.

Business context

- What does FEU stand for, and what is its scope?
- Which business or client does it serve, and what decisions consume its output?
- What is the cost of a bad forecast here — stockouts, waste, working capital, service penalties?
- Who are the consumers of the forecast, and what do they do with it?
- What is the planning cadence (weekly? monthly?) and the cut-off timing?

Data

- What are the source systems, and who owns each?
- What is the grain of the input data (item × location × day?), and the forecast output grain?
- How much history is available and used?
- What master data is involved, and what are the known quality issues?
- How are promotions, prices and calendars supplied?

Hierarchies

- Exact product hierarchy levels, top to bottom
- Exact customer/location hierarchy levels
- **FEU's definition of APG** — write it down verbatim
- At which level is the forecast generated? Aggregated? Agreed with the business?
- Top-down, bottom-up or middle-out?

Modelling

- What models are in production, and why those?
- What is the baseline definition, and how is uplift computed?
- What features are used?
- How and when is the model retrained?
- How are new products and discontinuations handled?
- How is intermittent demand handled?

KPIs

- **FEU's exact FA formula** — write out the numerator and denominator
- **FEU's exact FB formula** — and the sign convention
- At what level and lag are they reported?
- What are the current accuracy and bias figures, and the targets?
- Who reviews them, and how often?

Technical

- End-to-end flow from source to published forecast
- Which components run in ADF vs Databricks
- Storage layout and zone names

- Where the code lives; repo structure and branch policy
- How a change reaches production
- Where jobs are monitored and how failures are alerted

Operations

- The backtesting process — how it's run, what it reports, what "good" looks like
- Parallel run — duration, comparison metrics, exit criteria
- Common production failures and their runbooks
- Support rotation and escalation path

4. How to Use the KT Sessions Well

The sessions are the only source for the above, and Week 4's end-of-week activity says explicitly that the doubt-clearing session is meant to leave you "ready for final assessment."

- **Take notes in this structure**, not linearly. You'll immediately see which cells are empty.
- **Ask about definitions, not just process.** "How is FA calculated here?" is a better question than "what's your accuracy?" — and the answer is more useful.
- **Ask for one worked example end to end** — a single SKU through the whole pipeline. One concrete trace teaches more than three architecture diagrams.
- **Write down acronyms immediately**, with the speaker's own words. Every organisation has terms that mean something slightly non-standard.
- **Save the doubt-clearing sessions for genuine gaps**, not for things in the SharePoint docs — read those first.
- **Note who owns what.** Knowing whom to ask is itself the deliverable in a support context.

5. Assessment Preparation

Week 3 and 4 expected outcomes are all explanation-shaped: *able to explain FEU E2E flow, able to explain forecasting terminology used in FEU, able to explain hierarchy structures, understand FEU production support workflow.*

So prepare to **explain out loud**, not to recall definitions:

- ☐ Describe the FEU end-to-end flow in two minutes, source to consumption
- ☐ Draw the architecture from memory, naming each component's role
- ☐ Explain the product and customer hierarchies and at which level forecasting happens
- ☐ Define FA and FB using FEU's formulas, and say why bias matters
- ☐ Walk through what happens when a production run fails
- ☐ Explain what backtesting and parallel run each verify
- ☐ Trace one SKU from raw sales through to published forecast

If you can do all seven without notes, Weeks 3 and 4 are covered.